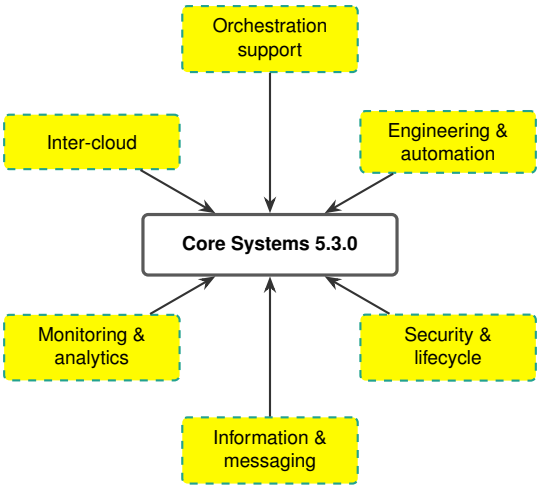
Arrowhead Support Systems 5.3.1 – White Box

Generic System of Systems Design Description



Abstract

This document is the white-box Generic System of Systems Design Description (GSoSDD) of the Eclipse Arrowhead 5.3.0 Support Systems. It covers the three support systems released upstream at v5.2.2 – **Blacklist**, **TranslationManager** and **DeviceQoSEvaluator** – and the twelve active project support systems migrated from Arrowhead 4.6 to the 5.2.2 baseline plus three monitor and management systems: LocalCloudManager, LocalCloudConfigurator and SoSConfigurator (with subsequent primary-source packaging described in Section 4.3): **PlantDescriptionEngine**, **Choreographer**, **Configuration**, **CertificateAuthority**, **Onboarding**, **EventHandler**, **DataManager**, **TimeManager**, **SysMonitor**, **AIGym**, **Gatekeeper** and **GateTunnel**. The deprecated SysMonitorPublisher is retained for historical traceability, not as an active support implementation. For each system it describes the internal design, produced and consumed services, persistent data, configuration and security, and it describes how the support systems depend on the Core and on each other. It is the companion of the white-box Core GSoSDD [1] and the white-box counterpart of the black-box *GSoSD Arrowhead Core and Support Systems 5.2.2* [2], which replaces the largely empty *GSoSD Arrowhead Support Systems 5.0* [3].

Contents

1 Overview **4**
1.1 Provenance and Scope . . . 4
1.2 Document Structure . . . 4
1.3 Significant Prior Art . . . 4
1.4 How This SoS Is Meant to Be Used . . . 5

2 SoS Functionalities and Properties **5**
2.1 Functional properties of the SoS . . . 5
2.2 Configuration of SoS properties . . . 5
2.3 Data stored by the individual microsystems . . . 6
2.4 Non-functional properties . . . 6
2.5 Stateful or stateless . . . 6

3 Important Delimitations **6**

4 SoS Architecture **7**
4.1 System Inventory . . . 7
4.2 Implementation Patterns . . . 8
4.3 Independent support deployments . . . 8
4.4 Complete selectable Java support distribution . . . 9
4.5 The sysMonitor Contract . . . 9
4.6 Dependency Graph . . . 10
4.7 Deployed Java Reference Cloud (24 October 2026) . . . 11
4.8 Support-System Paths in the Deployed Cloud . . . 13

5 Network Deployment **14**
5.1 From network contracts to a deployable topology . . . 14
5.2 Reference placement and network separation . . . 14
5.3 Endpoint, address and port plan . . . 15
5.4 Allowed communication paths . . . 15
5.5 Deployment configuration and operational evidence . . . 16
5.6 Commissioning and failure checks . . . 16

6 Support System White-Box Descriptions **17**
6.1 Orchestration Support (upstream v5.2.2) . . . 17
6.2 Engineering and Automation . . . 18
6.3 Security and Life Cycle . . . 19
6.4 Information and Messaging . . . 21
6.5 Monitoring and Analytics . . . 21
6.6 Inter-Cloud . . . 22

7 Services **24**
7.1 Produced Services . . . 24
7.2 Consumed Services . . . 25

8 Use Cases (White Box) **26**
8.1 Device Onboarding and Off-boarding . . . 26
8.2 Plant-Description-Driven Orchestration . . . 26
8.3 Workflow Execution by the Choreographer . . . 27
8.4 Monitoring Data Path . . . 27
8.5 Inter-Cloud Service Consumption . . . 27
8.6 Plant-Managed Dynamic Orchestration (Local Extension) . . . 28

9 Security **28**
9.1 Default Secure Profile and Development Exceptions 28
9.2 Security Model 29

10 Non-Functional Requirements as Realised **30**

11 Port Allocation **30**

12 Interoperability with the Java Reference Implementation **32**
12.1 Evidence and Current Deployment Scope 32
12.2 Contract and Security Requirements 33
12.3 Boundaries and Acceptance of Further Replacements 33

13 Implementation Observations and Open Issues **34**

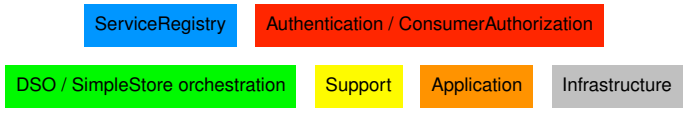
14 References **34**

15 Revision History **37**
15.1 Amendments 37
15.2 Quality Assurance 38

1 Overview

This document covers the Arrowhead support systems release-candidate v5.3.0

Graphics colour convention. All diagrams use the SysML v1.6 Papyrus library's system-role fills, taken from `EclipseArrowhead-5.0/Arrowhead-5.0.notation`: registry blue, authentication and authorisation red-orange, orchestration green, support yellow and application orange. Both orchestration implementations share the library's combined-cloud green; infrastructure is grey and mixed-system groups are neutral. These colours identify roles rather than implementation languages. The library's turquoise is reserved for service-interface elements.



1.1 Provenance and Scope

The support systems described here have two different origins, which must not be confused:

Upstream releases (Eclipse Arrowhead, v5.2.2 tags). **Blacklist** [4], **TranslationManager** [5] and **DeviceQoSEvaluator** [6] have been updated with a prodiuced sysMonitor service. Descriptions are based on the tagged `v5.3.0` sources.

Project migrations (EISLAB workspace). The 13 systems of `ah5-support-java-spring` [7]. They were migrated from the Arrowhead 4.6 core and support systems (or newly designed, as for **SysMonitor** and **AIGym**) onto the 5.3.0 Core and common library. Descriptions are based on the workspace sources, READMEs and the per-system SysD/SysDD/SD/IDD documents.

Several project migrations rely on an extension of `arrowhead-common` that exists only in the workspace: the native *sysMonitor* publisher (`eu.arrowhead.common.sysmonitor`). Where a statement depends on it, this is noted.

1.2 Document Structure

Section 1.3 references prior art, Section 1.4 the intended usage, Section 2 common properties and Section 3 the delimitations. Section 4 describes the SoS architecture: system inventory, implementation patterns, the *sysMonitor* contract and the dependency graph. Section 6 gives the white-box description of every support system. Section 7 lists produced and consumed services. Section 8 gives white-box use cases. Section 9 covers security, Section 10 non-functional properties, Section 11 port allocation and Section 13 implementation observations and open issues.

1.3 Significant Prior Art

- **Arrowhead 4.6 (`core-java-spring`).** Choreographer, Configuration, DataManager, EventHandler, TimeManager, PlantDescriptionEngine, Certificate Authority, Onboarding controller, Gatekeeper, Gateway and QoS Monitor were 4.6 systems sharing one SQL database. The 5.2 migrations keep their functionality and, where sensible, their URL paths, but give every system its own store and adopt 5.x identities, service operations and naming [8].
- **GSoSD Core 5.0** [9] and the white-box **Core GSoSDD 5.2.2** [1]: the Core services, filter chain, policies and orchestration approaches that all support systems build on.
- **External standards**: SenML (RFC 8428) [10] in **DataManager**, SNTP (RFC 4330) [11] in **TimeManager**, PKCS#10 (RFC 2986) [12] and X.509 in **CertificateAuthority** and **Onboarding**, AMQP 0-9-1 for *sysMonitor*, MQTT 5 for *sysMonitorData* and the inter-cloud relay.

1.4 How This SoS Is Meant to Be Used

Support systems are optional and deployed per need on top of a working Core local cloud. They behave like any other microsystem: they register in the **ServiceRegistry**, authenticate, are authorised by **Consumer Authorization**, and are found through orchestration. Six functional groups are distinguished (Table 1).

Group	Systems	Purpose
Orchestration support	**Blacklist**^{U}, **TranslationManager**^{U}, **DeviceQoSEvaluator**^{U}	Exclude systems, bridge incompatible interfaces/data models, rank providers by device QoS during dynamic orchestration.
Engineering & automation	**PlantDescriptionEngine**, **Choreographer**, **Configuration**	Derive orchestration rules from an engineered plant description; execute workflows; distribute configuration files.
Security & life cycle	**CertificateAuthority**, **Onboarding**	Admit and withdraw devices and systems; issue and check X.509 credentials.
Information & messaging	**EventHandler**, **DataManager**, **Time Manager**	Publish/subscribe events; cache and historise SenML data; trusted time.
Monitoring & analytics	**SysMonitor**, **SysMonitorPublisher** (deprecated), **AIGym**	Collect and store operational telemetry of all microsystems; experiment with models on it.
Inter-cloud	**Gatekeeper**, **GateTunnel**	Global service discovery, inter-cloud negotiation and tunnel/session control.

Table 1: Functional groups of the 5.3.0 support systems. "$Xystem$"^{U} = upstream v5.2.2 release; all others are EISLAB project migrations.

2 SoS Functionalities and Properties

2.1 Functional properties of the SoS

- **Orchestration enrichment** – blacklisting, protocol/data-model translation bridges and QoS-based filtering and sorting, invoked by **DynamicServiceOrchestration** when the corresponding `enable.*` flags are set.
- **Engineering-driven orchestration (management layer)** – **PlantDescriptionEngine** turns consumer/producer connections into managed **DynamicServiceOrchestration** provider rules and raises alarms on deviations; with the plant-description editor of the LocalCloudManager console, its first engineering UI, it forms the management layer of the local cloud.
- **Workflow execution** – **Choreographer** runs plans of actions and steps (a DAG per action) on registered executors, with orchestration made for each executor.
- **Admission control and credential life cycle** – **Onboarding** admits operator-approved devices and their systems step by step (onboarding, device and system certificates, identities) and off-boards them again; **CertificateAuthority** signs, checks and revokes end-entity certificates and manages trusted keys.
- **Data, events, time, configuration** – SenML proxy and historian, event publish/subscribe with metadata filtering, NTP-checked trusted time, per-system configuration files.
- **Monitoring** – every microsystem publishes *sysMonitor* observations (`echo`, `power`, `status`) over AMQPS; **SysMonitor** discovers, subscribes, stores and republishes them over MQTT; **PlantDescription Engine** and **AIGym** use them.
- **Inter-cloud exchange** – Gatekeeper global service discovery and inter-cloud negotiation, and GateTunnel sessions, carried between local clouds by an organisation-level MQTT relay with a name-constrained organisation PKI.

2.2 Configuration of SoS properties

In the Java-Spring reference implementation every support microsystem is configured through `application.properties`, with environment placeholders for deployment values and secrets from `/run/secrets`. Common switches are those of the Core (`authentication.policy`, `server.ssl.*`, `management.policy`, `enable.blacklist.filter`) plus:

- `service.authorization.enabled` (**TimeManager**, **DataManager**, **Configuration**, **Plant DescriptionEngine**): per-call *authorization* `verify` of produced application services, cached 30 s and failing closed. The secure default profile enables this enforcement. A component sample that sets it to `false` is a development exception; registration or orchestration alone does not authorise a provider call.
- `sysmonitor.enabled`, `sysmonitor.broker.*`, `sysmonitor.ssl.*`, `sysmonitor.local.cloud`, `sysmonitor.publish.interval` (default 30 s): the native *sysMonitor* publisher.

2.3 Data stored by the individual microsystems

Every support system owns its data (Table 2); none shares a schema with another system. The micro-systems use their own MySQL schema, accessed with MariaDB Connector/J (`jdbc:mariadb`, TLS with full verification). Each system's database account holds data-manipulation privileges on its own schema only; schema creation is reserved for the initialisation scripts (Section 9.2). **AIGym**, **LocalCloudManager** and **SysMonitorPublisher** keep only volatile state.

2.4 Non-functional properties

- **Security**: full AA and encrypted information exchange are the default from v5.0. The Java reference realises and extend this through mutual TLS, `OUTSOURCED` authentication with identity tokens bound to the certificate CN, `CERT_AUTH` interface policy, ConsumerAuthorization grants per service, and `sysop-only` management. Every system refuses to start with an insecure configuration unless a development profile is selected explicitly.
- **Observability**: uniform liveness, power and status telemetry with `alive`/`degraded` semantics tied to each system's own health (database, NTP trust, synchronisation alarms).
- **Robustness**: failure of a support system never stops the Core; orchestration features degrade to warnings or, where forced, fail closed.
- **Scalability**: all systems are designed for allowing multiple instance; replication between multiple instances will need its dedicated implementation no shared stores between microsystems are allowed.

2.5 Stateful or stateless

All systems except **AIGym** (ephemeral models) and **SysMonitorPublisher** are stateful. Long-lived state: blacklist entries, translation bridges, device statistics, plant descriptions and alarms, plans and sessions, configuration files, issued certificates and trusted keys, event subscriptions, SenML history, observation history, cloud/relay records and tunnel sessions. **TimeManager** keeps only logs.

3 Important Delimitations

- Upstream systems are described at their `v5.2.2` tags; workspace modifications of those repositories (removal of the *monitor* service, native *sysMonitor*) are noted and part of the `v5.3.0` release.
- Inter-cloud exchange is provided by the project implementations of **Gatekeeper** and **GateTunnel** over the inter-cloud relay (Section 6.6.3), between local clouds. The upstream 5.2.2 release has no counterpart for **DynamicServiceOrchestration**'s `enable.intercloud`.
- Human identity federation, automated OCSP, ACME/EST/SCEP and automatic CA key rotation are not provided. CRL publication and configured enforcement are implemented; operator console password sessions are a separate mechanism.
- **LocalCloudManager**, the operator tool of the test clouds, is a client of these systems and described in this document.

4 SoS Architecture

4.1 System Inventory

Microsystem	Port	Technology	Persistence	Origin
Blacklist^{U}	8464	Java 21, Spring Boot	MySQL `ah_blacklist`	New in 5.x
TranslationManager^{U}	8465	Java 21, Spring Boot	MySQL `ah_translation_manager`	New in 5.x (successor of the 4.x Translator)
DeviceQoSEvaluator^{U}	8472	Java 21, Spring Boot, Quartz; Python device agent	MySQL `ah_device_qos_evaluator`	New in 5.x (successor of 4.6 QoS Monitor)
PlantDescriptionEngine	8432	Java 21, Spring Boot	MariaDB `ah_plantdescriptionengine`	4.6 PlantDescriptionEngine
Choreographer	8457	Java 21, Spring Boot	MariaDB `ah_choreographer`	4.6 Choreographer (core)
Configuration	8459	Java 21, Spring Boot	MariaDB `ah_configuration`	4.6 Configuration (core)
CertificateAuthority	8448	Java 21, Spring Boot, Bouncy Castle	MySQL `ah_certificate_authority`	4.6 Certificate Authority
Onboarding	8435	Java 21, Spring Boot 3.4.9	MariaDB `ah_onboarding` (enrolments, logs)	4.6.2 Onboarding controller; procedure of [13]
EventHandler	8455	Java 21, Spring Boot	MariaDB `ah_eventhandler`	4.6 EventHandler (core)
DataManager	8461	Java 21, Spring Boot	MariaDB `ah_datamanager`	4.6 DataManager
TimeManager	8463	Java 21, Spring Boot	MariaDB `ah_timemanager` (logs)	4.6 TimeManager (core)
SysMonitor	8475	Java 21, Spring Boot, AMQP intake, MQTT output	MySQL `ah_sysmonitor`	New (5.0 draft SysD “MicrosystemMonitor”)
SysMonitorPublisher	–	container sidecar	none	New; deprecated 2026-10-11
AIGym	8478	Java	in memory	4.6 `rsafg-ai-gym-client`
Gatekeeper	8449	Java 21, Spring Boot, MQTT relay client	MySQL `ah_gatekeeper`	4.6 Gatekeeper
GateTunnel	8453	Java 21, Spring Boot, MQTT relay client	MySQL `ah_gatetunnel`	4.6 Gateway
LocalCloudManager	8491 (console 8490)	Java 21, Spring Boot; separate Docker proxy	none	Workspace (formerly Node.js/nginx)
LocalCloudConfigurator	8492 (host worker 8493)	Java 21, Spring Boot	MySQL `ah_localcloudconfigurator`	Workspace (port of the Go LocalCloudConfigurator)
SoSConfigurator	8494 (console 8495)	Java 21, Spring Boot, ColonyOS client	MySQL `ah_sosconfigurator`	Workspace

Table 2: Inventory of the 5.2 support systems (canonical ports, Section 11). ^{U} = upstream v5.2.2 release.

4.2 Implementation Patterns

4.2.1 Spring systems on `arrowhead-common`.

The Java-spring microsystems share the internal architecture described for the Core in [1]: `SystemInfo` declaring system and service models, `ApplicationInitListener` performing SR registration, the ordered filter chain (authentication → blacklist → management), validation and normalisation, a service layer, engines/drivers, JPA persistence in a private schema where the system stores data, and Quartz or scheduled jobs where it has periodic works, and *generalManagement* (`get-log`, `get-config`). The workspace common library now advertises `CERT_AUTH` for HTTPS interfaces by default (23 October 2026); some services retain explicit registration overrides. This is distinct from the released library's older `NONE` behaviour.

4.2.2 Systems converted to the Java-Spring pattern

Until 8 November 2026 seven support systems did not follow the pattern above: **CertificateAuthority**, **Gatekeeper**, **GateTunnel** and **SysMonitor** were native JDK programs (`HttpsServer`, SQLite), **LocalCloud Manager** was a Node.js API behind nginx, and **LocalCloudConfigurator** and **SoSConfigurator** were plain JDK HTTP servers. All seven now use the common library, filter chain, registration, security profile guard, *general Management* and *sysMonitor* publisher of the other Spring systems, and those that store data use a private MySQL schema. Service definitions, operations, HTTP paths (including the former unprefixed aliases), JSON documents, the AH-GT-1 relay protocol and the *sysMonitor* and *sysMonitorData* formats are unchanged; the conversion is verified by route contract tests and end-to-end tests against MySQL (Section 4.3). The Python RandomDataProducer is an application, not one of these support backends [14].

4.3 Independent support deployments

Every support system is independently buildable and deployable from its own folder in the Eclipse-Arrowhead distribution. Each folder owns its source, private common-library copy (`vendor/`), Maven POM, tests, Spring Boot JAR, Dockerfile, environment template and Compose definition. No build context or source import points to a sibling support system. No common support runtime image, runtime role selector, shared database or common configuration bundle is required.

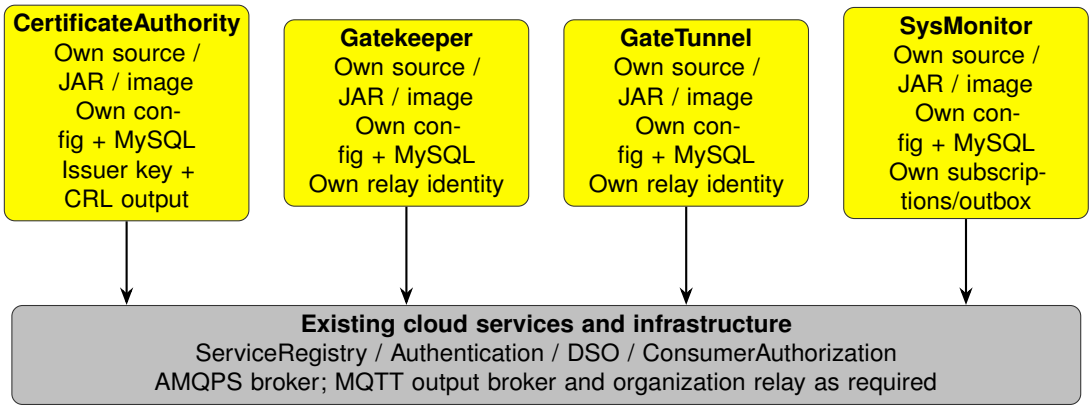


Figure 1: Independent support packaging and private state. Arrows denote network service use, not shared implementation or data. Operators select the support systems needed by their local cloud.

Containers run as UID 10001 with a read-only root filesystem and bounded non-executable temporary storage. Only CertificateAuthority mounts the issuer signing key and writes the published CRL; the other systems receive public trust material, their own identity and a read-only CRL. CRL distribution is an explicit cloud operation. CertificateAuthority and Gatekeeper can import a store of their former native implementation into the new schema.

The operator consoles of **LocalCloudManager** (8490) and **SoSConfigurator** (8495) are separate plain-HTTP connectors with password login (scrypt hash, HttpOnly session cookie, per-client lockout, Host and Origin checks), published on the host's loopback interface only; browsers need no client certificate. Their Arrowhead

services use mutual TLS like every other system. **LocalCloudManager** reaches Docker only through a separate Docker-proxy process with a fixed allowlist (container list, logs and start/stop/restart of the project's managed containers). The host worker of **LocalCloudConfigurator** and the executors of **SoSConfigurator** are non-web Spring applications of the same JARs: they are agents of their configurator, not cloud systems, and are not registered.

Independence concerns packaging and state ownership. ServiceRegistry remains the bootstrap service; Authentication and DynamicServiceOrchestration are discovered there, and ordinary consumption follows DSO and authorization rules. AMQPS monitoring, SysMonitor's MQTT output and federation's organization MQTT relay remain network dependencies. Gatekeeper and GateTunnel cooperate for inter-cloud service use, without forcing a combined artifact or process. AH-GT-1 is supported and selected in the federation Compose samples; compatibility with legacy-mTLS is explicit and does not permit silent downgrade.

The per-system SysDDs specify these build and deployment boundaries. Verification uses isolated builds, unit, security and route contract tests, end-to-end tests against a fresh MySQL schema with the restricted database account, secure-profile startup checks and, for the relay systems, round trips against a local TLS broker. This document describes deployable packaging and does not assert a new live-cloud deployment.

4.4 Complete selectable Java support distribution

The standalone distribution covers all nineteen support systems. Each project owns its application source, executable JAR, Dockerfile, configuration, tests and Compose definition. The standalone support project builds do not read JavaSupportRuntime or a sibling support project. Existing cloud deployment consumers may retain historical build contexts until deliberately retargeted.

The ten database-backed Spring systems (Blacklist, TranslationManager, DeviceQoSEvaluator, Onboarding, DataManager, TimeManager, EventHandler, PlantDescriptionEngine, Configuration and Choreographer) and, since 8 November 2026, CertificateAuthority, Gatekeeper, GateTunnel, SysMonitor, LocalCloudManager, LocalCloudConfigurator and SoSConfigurator compile private Arrowhead common utilities and DTO source under `vendor/`. Their POMs inherit only the published Spring Boot parent; no Arrowhead common/core parent or preinstalled Arrowhead library is required. Local provenance manifests preserve source hashes, licensing and upstream POMs. Source security fixes must be propagated to every affected private copy. AIGym keeps its existing independent Spring Boot build.

Each database-backed deployment starts its own MySQL instance and initializes only its own schema on a fresh project volume. The DB has no host port, has an operator-selected private subnet and requires TLS. Application and database passwords are independent file secrets. App credentials and trust are mounted individually; the app sees the public database CA but no DB key or CA signing key. The production configuration requires mutual TLS, outsourced authentication, certificate/token binding and a valid CRL. Containers use an unprivileged application user and a read-only root filesystem. Database credentials and schemas are not shared across selected support systems.

Packaging independence preserves network contracts. ServiceRegistry, Authentication, DSO, Consumer-Authorization and required brokers/providers must be provisioned in the selected cloud. Some systems refuse startup until core services are available. Deployment tests therefore use isolated mutual-TLS registry/authentication/DSO protocol fixtures, own test credentials and private databases, with one support system selected at a time. Authorized HTTPS requests and missing-token/certificate rejection are checked. Listener checks alone do not establish application, monitoring, federation or NTP readiness; the distribution verification report states exact coverage and limitations.

The copied core project and historical combined runtime are not support-system deployment units in this inventory.

The distribution `DEPLOYMENT-GUIDE.md`, each project `DEPLOYMENT.md` and the all-system verification report specify project names, host ports, private subnets, credential provisioning, build commands and state lifecycle. The per-system SysDDs describe the new packaging. Existing live-cloud evidence remains historical baseline; this packaging change does not redeploy live clouds.

4.5 The sysMonitor Contract

Monitoring is the cross-cutting concern that ties the support systems together (Figure 2).

- Every microsystem produces a *sysMonitor* 1.0.0 service with interface `generic_amqps`, policy `CERT_AUTH` and metadata `publisherMode=native`; the `generic_amqp`/`generic_amqps` interface templates are created in the **ServiceRegistry** at deployment.
- Each operation is the periodic publication (default every 30 s) of one observation type to the AMQP topic exchange `arrowhead` of the cloud's AMQP broker (RabbitMQ 4.0, AMQP 0-9-1 over TLS, port 5671) with routing key `<cloud>.sysmonitor.<System>.<serviceInstance>.<type>`; baseline types are `echo` (`alive` or `degraded`), `power` and `status`. Messages are persistent and confirmed by the broker.
- Publishers authenticate with their own certificate (SASL `EXTERNAL`: the broker user is the certificate CN). The broker's access list names, per CN, the routing keys a system may publish and the binding keys it may bind its own queues with; every system may publish only below its own routing key, and **SysMonitor** may bind `<cloud>.sysmonitor.*.*.*`.
- The observation envelope carries `schemaVersion`, `observationId`, `providerSystem`, `serviceInstance`, `observationType`, `sourceTime`, a monotonic `sequence`, `value`, `unit`, `quality` and `metadata` [15].
- **SysMonitor** needs a ConsumerAuthorization grant `<System>|sysMonitor` for every provider. Publishers are the common-library `SysMonitorPublisher` (Java systems), the RabbitMQ Java client (native Java and **AIGym**), the Java **ServiceRegistry**, pika (RandomDataProducer) and amqplib (LocalCloudManager).
- The AMQP broker carries only *sysMonitor*. The MQTT broker (Mosquitto 2.0, port 8883) stays in place for *sysMonitorData* and all other intra-cloud MQTT traffic.
- HTTPS liveness probes, where still present, are registered as service *echo* (never as *monitor*, *monitoring* or *sysMonitor*).

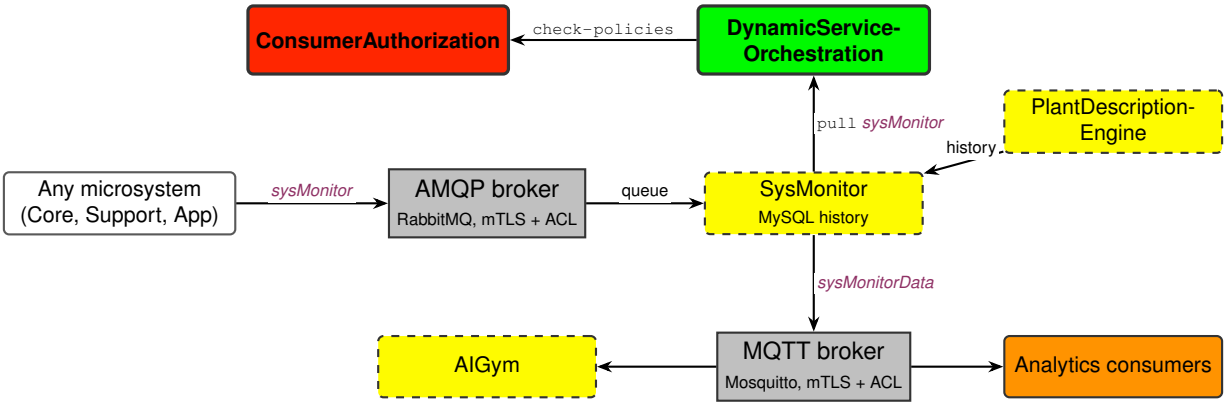


Figure 2: Monitoring chain: observations over AMQP into SysMonitor, history over HTTPS, normalised stream over MQTT.

4.6 Dependency Graph

Figure 3 shows which Core and Support services each support system consumes. PDE, Choreographer, SysMonitor and AIGym use DynamicServiceOrchestration. SimpleStore remains an independent optional system; it is no longer a dependency of PDE. The local managed-plant API and schema upgrade are described in Section 8.6.

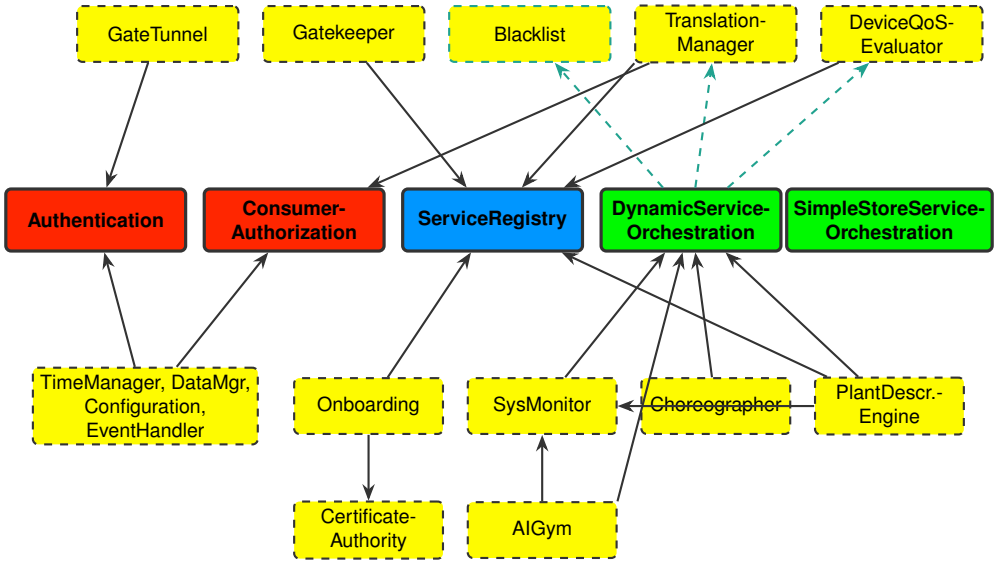


Figure 3: Consumption dependencies of the support systems (dashed: invoked by `DynamicServiceOrchestration` when enabled). Registration in the ServiceRegistry and identity/authorisation calls of every system are omitted for clarity.

4.7 Deployed Java Reference Cloud (24 October 2026)

The local Java reference deployment is the Docker Compose project `arrowhead-v52-test-cloud`, maintained in `Arrowhead-test-cloud-java`. Figure 4 records its 22 deployed microsystems, including both orchestration variants, the operations manager and the demonstration application [14]. This is a dated deployment example of the architecture, not a requirement that every local cloud deploy every optional support system. In this 24 October snapshot, the five Core systems and fifteen support systems are Java implementations; **LocalCloudManager** has a Node.js API and **RandomDataProducer** is a Python application. Thus “Java cloud” identifies the reference implementation baseline, not the language of every process.

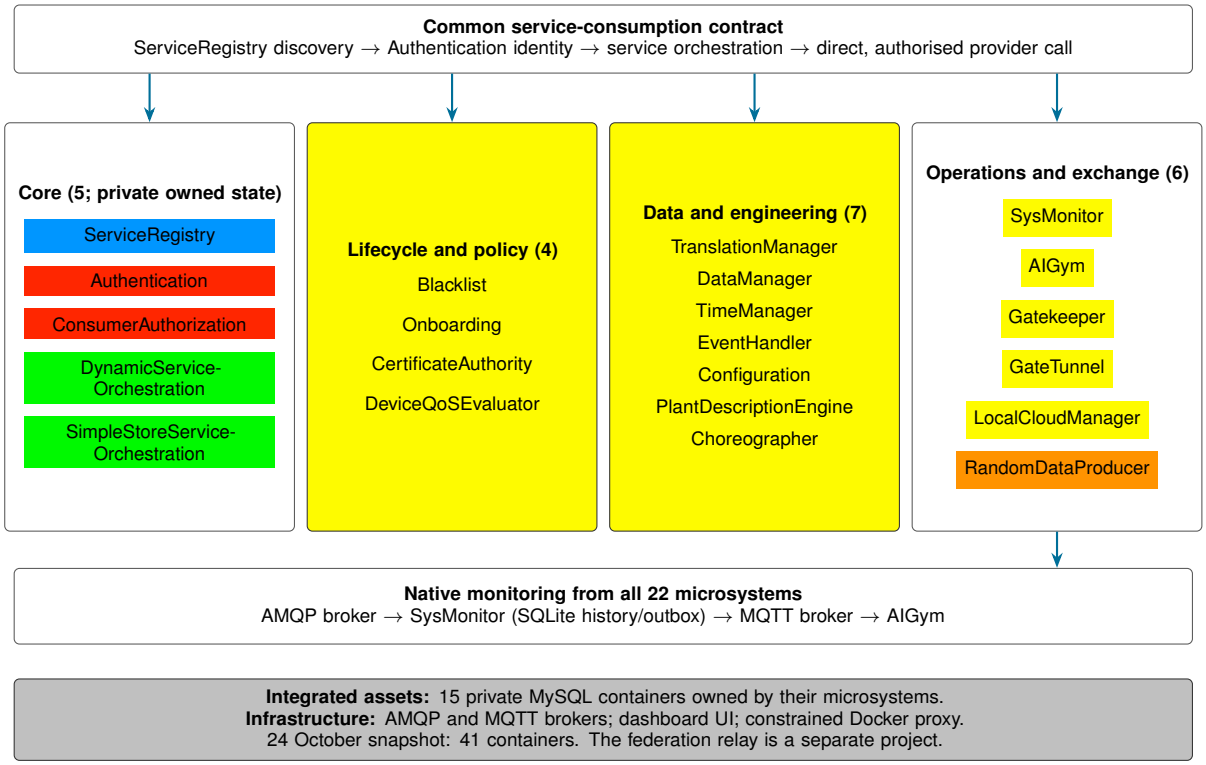


Figure 4: Deployed Java reference cloud. All 22 named systems are microsystems; message brokers, dashboard and Docker proxy are infrastructure; database containers are integrated assets inside their host microsystems. The shared consumption path is drawn once to avoid repeating bootstrap edges.

The 41 running containers comprise the 22 microsystems, 15 dedicated MySQL containers, two message brokers, a dashboard UI and a Docker proxy. The four **CertificateAuthority**, **Gatekeeper**, **GateTunnel** and **Sys Monitor** services use independently packaged Java 21 implementations, each retaining its own SQLite state. The former delegated **SysMonitorPublisher** is not a deployed microsystem: each system publishes its own *sys Monitor* observations.

Discovery, orchestration and direct consumption. Systems bootstrap through **ServiceRegistry**, locate *identity* and *serviceOrchestration*, obtain a certificate-bound identity token and request an authorised provider for the required operation. The resulting call goes from consumer to provider; the orchestrator does not proxy the application payload. DSO directly discovers its own authorisation and blacklist dependencies to avoid recursive orchestration. Authentication also bootstraps its authorisation check. Other dependency calls use orchestration, with the documented caches and exceptions.

Monitoring and application data. Native publishers send observations over AMQPS to RabbitMQ; **Sys Monitor** consumes authorised selections and stores them in its own SQLite history/outbox. It republishes normalised *sysMonitorData* through MQTT for **AIGym**. **LocalCloudManager** reads *sysMonitorHistory* and manages subscriptions. The demonstration producer instead writes application SenML data to **DataManager** every 30 s through *dataHistorian* `put-data`. Monitoring history is not stored by DataManager.

Engineering, workflow and federation paths. The plant editor calls **PlantDescriptionEngine**; an active plant is compiled into an atomic DSO rule snapshot through *serviceOrchestrationPlantManagement*. **Choreographer** asks DSO for push orchestration on behalf of workflow executors; no executor is deployed by default. Onboarding consumes certificate signing, validation and revocation, identity management and blacklist operations. Inter-cloud selection uses Gatekeeper discovery/negotiation and an AH-GT-1 GateTunnel session over the external

relay. The 24 October deployment snapshot contained neither SimpleStore route rows nor an active DSO plant-rule row: these mechanisms are available, but the diagram does not imply an active engineered workflow.

Reference security profile. The deployed service profile realises the v5.0-and-later default: full AA and information encryption, using mutual TLS with `CERT_AUTH`, certificate-bound identity tokens and operation-scoped authorisation. Unsecured component samples are explicit development exceptions, not an alternative architectural default. The operator console requires login and reaches Docker only through a project-scoped proxy. CertificateAuthority publishes CRLs; services and brokers enforce them on new TLS handshakes. Ending established connections is a separate action. AH-GT-1 protects relayed payloads end to end; cloud application trust remains separate from relay/organisation trust [14, 16].

4.8 Support-System Paths in the Deployed Cloud

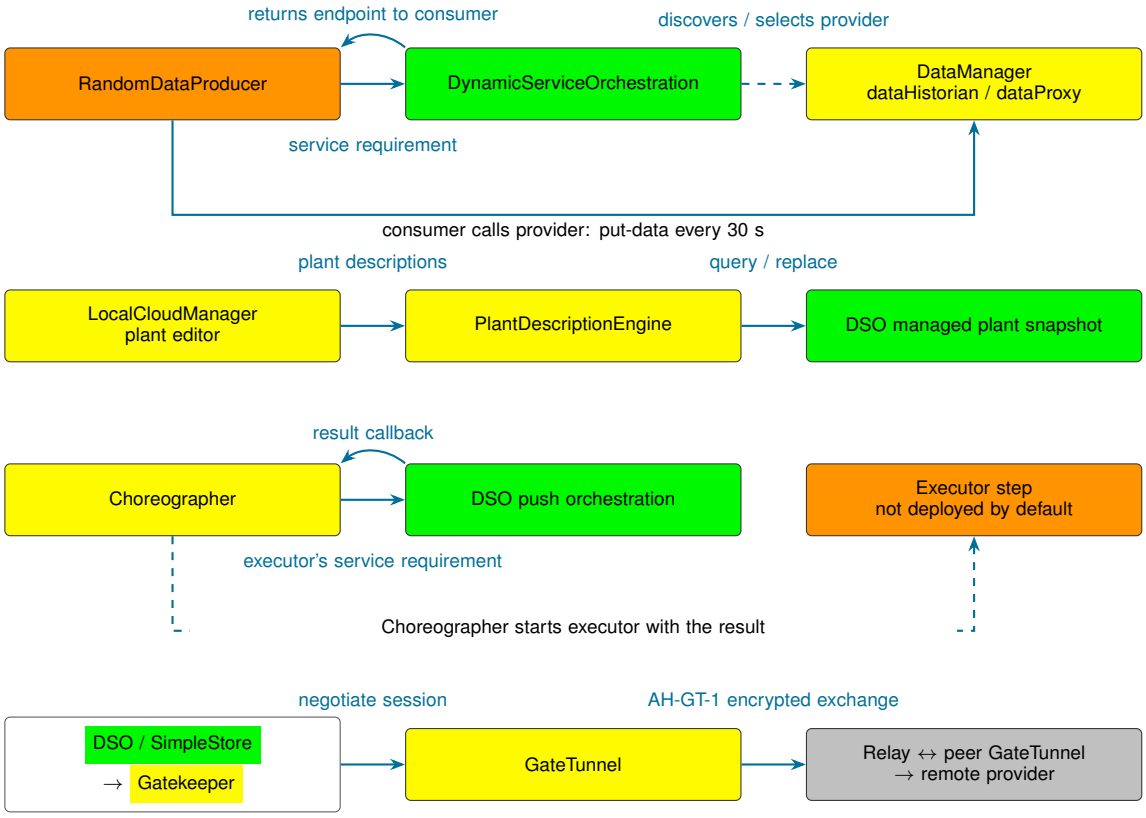


Figure 5: Main support-system integrations: orchestrated application storage, plant-to-DSO synchronisation, push-orchestrated workflows and encrypted federation. The workflow executor is not deployed by default.

LocalCloudManager orchestrates its DataManager, SysMonitor, PDE, SimpleStore, DSO-history, authorisation-policy, federation-status and producer-status calls. TimeManager, EventHandler and Configuration have service-specific consumer policies. DeviceQoSEvaluator and TranslationManager remain independently registered support providers; the Java inventory does not imply a deployed translator, device agent or workflow executor. AIGym consumes normalised monitoring, while PDE can read monitoring history as well as manage DSO plant scopes. These paths retain each provider's own authentication and authorisation checks after orchestration.

5 Network Deployment

5.1 From network contracts to a deployable topology

The deployment view realises the intra-cloud, inter-cloud and DMZ interface contracts defined by the `Network` model in `Arrowhead-5.2.uml` [17]. It follows the GSoSD Network Architecture section: the model supplies contract/specialisation structure and Ethernet/WiFi examples, whereas the deployment must supply devices, address plans, routes, service bindings and enforcement points. No actual VLAN, subnet, firewall or device allocation is asserted by importing the library.

The source's `GenericIntraCloudNetwork-DesignDescription` refines the intra-cloud contract with an Ethernet-typed port. `GenericIntercloudNetwork` refines the inter-cloud contract with a WiFi-typed port. `GenericIntracloudNetworkDMX` refines the DMZ contract with an Ethernet-typed port. These ports subset their generic full-port interfaces; their 802.3/802.11a names describe the model examples rather than a mandatory physical layout. The source DMX spelling is not interpreted as a separate bearer.

This Support deployment adds only the segments and flows required by the selected support services. Local monitoring, engineering, onboarding and inter-cloud federation have different dependencies. Gatekeeper/GateTunnel and a DMZ are needed only when the chosen federation design calls for them; deploying a local historian or monitor does not by itself require a federation relay.

5.2 Reference placement and network separation

Figure 6 gives an explicit reference interpretation of the library, not a record of a running cloud. An intra-cloud segment can be a physical LAN, a routed/segmented LAN or an isolated virtual/container network. The `Device` model contains a generic device design, but it does not allocate the network classifiers to an actual host inventory. The deployment therefore records which hosts run which processes and which interfaces attach them to the selected segments.

Externally exposed federation endpoints may be placed on a controlled DMZ segment and given only the internal paths needed by their Gatekeeper/GateTunnel roles. A deployment without a separate DMZ must document how an equivalent boundary is enforced. The model does not specify that a component is dual-homed, nor does an interface classifier automatically configure forwarding between its host interfaces.

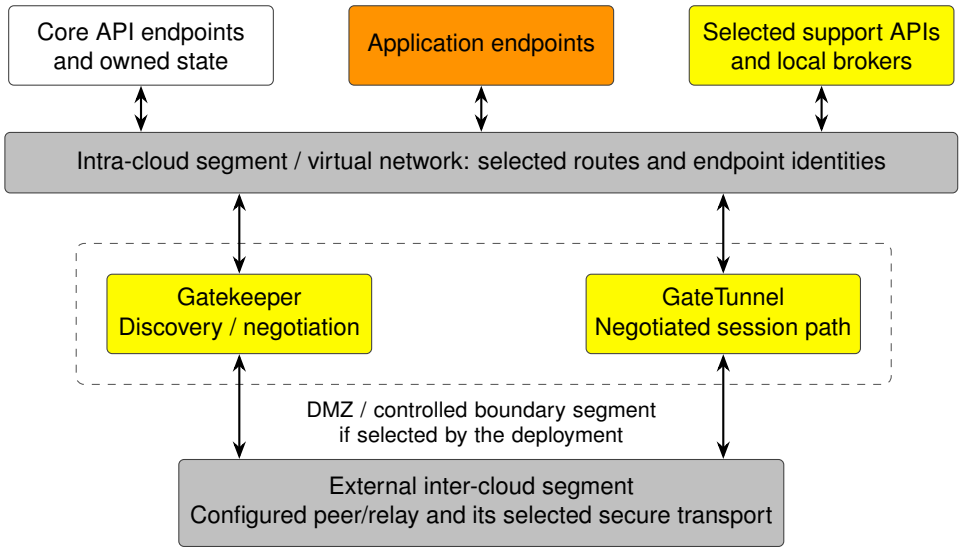


Figure 6: Reference network deployment derived from the model's interface roles. Segments, boundary placement and enforcement are deployment refinements. Private stores remain assets of their owner microsystems; the figure grants no public database or management access.

5.3 Endpoint, address and port plan

For every selected interface the deployment record identifies its owning microsystem, process/host, network attachment, bind address/listening port, advertised address/port, transport and expected peer identity. Listening and advertised endpoints may differ behind host/container port publication or NAT; test the advertised endpoint from the consumer's network, not only from the provider host. Keep name resolution and certificate SAN/peer-identity checks consistent with that endpoint.

Use the port allocation in Section 11 together with the actual primary implementation and its per-system deployment configuration. Framework-system ports, broker ports and dynamically allocated session/callback endpoints have different scopes. Where the established cloud-index scheme is used, record host mappings and test collisions in the shared address namespace; do not blindly shift every internal listener or treat every broker as part of the 8000–8999 block. Do not open the whole allocation block merely to avoid defining the required flows.

Ethernet/WiFi configuration is a bearer decision. The service binding separately selects HTTPS, MQTT over TLS, AMQP over TLS or another implemented protected transport. `ProtocolKind`'s HTTP defaults and its CoAP/Websocket/DDS/Other alternatives are not evidence of unencrypted production operation or implementation support. Keep the existing telemetry and federation wire contracts unchanged when mapping them onto a segment.

5.4 Allowed communication paths

Initiator and destination	Allowed purpose / binding	Boundary requirement
Members to Core APIs	Registration, identity, authorisation and selected orchestration over the implemented protected API.	Reachable bootstrap endpoints; authenticated identities and required operation grants. No peer-cloud access to internal Core management by default.
Consumer to provider	Direct application/support calls to the discovered/orchestrated interface, or explicit approved callbacks.	The returned address must be reachable from that consumer. Validate provider identity and consumer authority independently of routing.
Publishers or consumers to local brokers	Native monitoring or other selected publish/subscribe contracts.	Separate configured broker endpoints, certificate identities and exchange/topic ACLs. A subscription grants no unrelated management or federation access.
Gatekeeper / **Gate Tunnel** to peer or relay	Discovery, negotiation and negotiated session traffic of the chosen inter-cloud binding.	Restrict DMZ/external flows to declared peers, relay endpoints and session policy; retain the cloud's internal authorisation checks.
Approved operators/engineering clients to management	Only the configured management or console connector and its selected operations.	A browser console session and an Arrowhead system identity are distinct. No blanket external exposure of management, runtime-control or database interfaces.
Microsystem to owned store	Private persistence connection inside its deployment boundary.	Store connectivity belongs to the owning system; no direct application-consumer or remote-cloud database route is implied.

5.5 Deployment configuration and operational evidence

Record the selected physical/virtual segments and, where used, VLAN/VRF identifiers, routes, firewall/ACL directions, NAT/host mappings, DNS names and the equipment/host ownership. Record the implemented TLS/identity profile, trust distribution and revocation enforcement points. Network isolation supplements authentication and authorisation; it cannot replace them. Provision any required name/time/credential-management dependencies explicitly without inferring them from a model connector.

In a container deployment, distinguish a private Compose/virtual network from a port published on a host interface. A private database network or a loopback-only operator console must not become publicly routable because the host also serves a federation endpoint. The selected primary Java, Go or Rust implementation supplies its actual defaults, startup checks and supported transports; a library port or a similarly named process does not prove that all variants provide the same security or protocol behavior.

The existing dated reference-deployment discussion in Section 4.7 remains a separate example. This network plan neither refreshes that runtime inventory nor claims that its historical images, routes or permissions have been changed. New deployment evidence must identify the exact source/build, effective configuration and observed endpoints.

5.6 Commissioning and failure checks

Before accepting a network realisation:

1. Verify each deployed network role against its model contract, host/interface allocation and effective address/port configuration. Record which DMZ/bearer refinements were selected and why.
2. Exercise bootstrap and authorised direct consumption from the intended consumer network, including advertised endpoints, required callbacks and configured broker paths. A local health request alone is insufficient.
3. Check that unapproved external, peer-cloud and management paths are refused. Test incorrect or untrusted peer identities and absent/denied authorisation separately from transport reachability; check revocation only at the enforcement points actually provided by the selected profile.
4. For selected federation, establish and close a session through the permitted peer/relay path; confirm that unrelated Core, management and store endpoints remain inaccessible across the boundary.
5. Interrupt an intra-cloud, DMZ or inter-cloud path and observe the documented timeouts/recovery. Verify restored reachability and security before resuming dependent exchanges; no redundant path or availability guarantee follows merely from the network class names.
6. Keep the endpoint/route/ACL observations and negative checks with the deployment record. Distinguish link/routing failure, TLS identity failure, authorisation refusal and application failure in that evidence.

These are acceptance obligations for a deployment, not tests executed by the documentation update. No production address, firewall rule, network device or running cloud is changed here.

6 Support System White-Box Descriptions

6.1 Orchestration Support (upstream v5.2.2)

6.1.1 Blacklist

Purpose. Cloud-level exclusion of named systems, used by the blacklist filter of every `arrowhead-common` system and by **DynamicServiceOrchestration** candidate filtering.

Services. *blacklistDiscovery* (`check`, `lookup`), *blacklistManagement* (`query`, `create`, `remove`), *generalManagement*. The v5.2.2 tag also registers a *monitor* service (`echo`); the workspace removes it in favour of native *sysMonitor*.

Internal design. An *entry* (system name, optional expiry, active flag, issuer, revoker, mandatory reason $\leq$ 1024 characters, audit times) is *in force* when active and not expired; a system is blacklisted while at least one entry is in force. `create` rejects the requester's own name and names on the configured `whitelist` (default `Blacklist`), and whitelisted names are inactivated at start-up. `remove` inactivates all active entries for the names (self-removal only by sysop). Entry access is serialised in `EntryDbService`. An `InternalBlacklistFilter` rejects (403) every request of a blacklisted requester except sysop requests, `lookup` and a `check` of its own name, so a blacklisted system can learn why it is excluded.

6.1.2 TranslationManager

Purpose. Creates *translation bridges* that let a consumer use a provider with a different interface (protocol) and/or data model, via *interface translators* and *data model translators*.

Services. *translationBridge* (`discovery`, `negotiation`, `abort`), *translationBridgeManagement* (`discovery`, `negotiation`, `abort`, `query`), *translationReport* (`report`), *generalManagement*.

Internal design. Layers: API → application services → `TranslatorBridgeEngine` → drivers (`CoreSystemsDriver`, `InterfaceTranslatorDriver`, `DataModelTranslatorFactoryDriver`) → persistence (`bridge_header`, `bridge_discovery`, `bridge_details`). Bridge states: `NEW` → `DISCOVERED` → `PENDING` → `INITIALIZED` → `USED`/`CLOSED`, or `ERROR`/`ABORTED`.

1. **Discovery**: authorisation checks of the consumer, candidates and translators under the default secure profile, plus configured blacklist checks; removal of candidates that need no or cannot get translation; lookup of interface translators (*interfaceBridgeManagement*) registered for at least `translator.service.min.availability` minutes; `check-targets` on each translator (with tokens); if data models differ, lookup of data model translators/factories (`check`); results stored as discovery data.
2. **Negotiation**: selection of one translator and common interface, initialisation of data model translators (`initialize`), a token allowing the translator to call the target, then `initialize-bridge`. The returned bridge interface must use policy `TRANSLATION_BRIDGE_TOKEN_AUTH`, else the bridge is aborted.
3. **Reports**: accepted only from the bridge's own translator for valid transitions (`USED` increments a counter; `INTERNAL_CLOSED`, `INTERNAL_ERROR`, `EXTERNAL_ERROR`).

TranslationManager is in **ConsumerAuthorization**'s `unbounded.token.generation.whitelist` and in the SR direct-access list. Discovery flags in management requests are honoured only if `allow.discovery.flags=true`.

6.1.3 DeviceQoSEvaluator

Purpose. Evaluates the quality of the *devices* hosting candidate providers so that **DynamicServiceOrchestration** can filter or sort them.

Services. *qualityEvaluation* (`filter`, `sort`), *deviceQualityDataManagement* (`query`, `reload`), *general Management*.

Internal design. Quartz jobs: `MeasurementOrganizerJob` pages through *serviceRegistryManagement* `system-query` and groups systems with overlapping addresses into devices (`SystemDeviceMap`); a device is *augmented* if a system's metadata `qos.deviceAugmented` names agent metric groups. `RttMeasurementJob` times TCP connects (`rtt.measurement.timeout`); `AugmentedMeasurementJob` reads `GEThttp://<device>:59473/device-qos` from the Python device agent (CPU load, memory, network ingress/egress). Statistics (min, max, mean, median, current) are stored per metric. `StatisticsEngine` scores each provider as $\sum_i w_i s_i$ over the evaluation window (`evaluation.time.window`, 60 s), with statistics scaled 0–100 and missing data counted as worst (100); `filter` keeps scores $\leq$ threshold, `sort` orders ascending. A cleaning job removes raw data and inactive devices.

6.2 Engineering and Automation

6.2.1 PlantDescriptionEngine

Purpose. Keeps the local cloud's orchestration consistent with an engineered *plant description* (systems, ports, connections) and raises alarms on deviations.

Services. *plantDescriptionManagement* (`query`, `get`, `create`, `replace`, `update`, `remove`), *plant DescriptionMonitor* (`query-plant-descriptions`, `get-plant-description`, `query-alarms`, `get-alarm`, `acknowledge-alarm`), *generalManagement*, *sysMonitor*.

Internal design. One description is active at a time, with its includes; v4.6 validation rules apply. A plant system matches a registered system when names are equal (if given) and its metadata is a subset of the registered metadata. A connection yields **DynamicServiceOrchestration** managed provider rules, one per producer service instance of the port's service definition, only when both ends match exactly one system; colliding priorities move to the next free one. A reconciliation loop runs every `plant.sync.interval` (15 s) and after each change, using *serviceRegistryManagement* (`system-query`, `service-query`) and *serviceOrchestration PlantManagement* (`query`, `replace`). Alarms: `SYSTEM_NOT_REGISTERED`, `MULTIPLE_MATCHES`, `SYSTEM_NOT_IN_DESCRIPTION`, `NOT_MONITORED`, `NO_RECENT_OBSERVATION` (the last two from *sysMonitor History*, stale after 120 s); alarms clear automatically and are acknowledged by operators. `echo` reports `degraded` while synchronisation fails or alarms are active. Data: `plant_description`, `pde_alarm`, `logs`.

Management layer and engineering UI. The PDE is the enforcement point of the management layer of the local cloud (GSoSD [2], Section 4.1). Its first engineering user interface is the **PD** view of the LocalCloudManager console (Spring console connector on 8490, Arrowhead API on 8491):

- a table of the stored plant descriptions (identifier, name, active, includes, number of systems and connections, last update);
- a form editor – name, includes, activation; systems with their ports (service definition, consumer or producer, metadata); connections from consumer port to producer port with priority – and an equivalent JSON editor for the complete plant-description document;
- Save and Delete, which the console API forwards as `create`, `replace`/`update` and `remove` of *plantDescriptionManagement*, authenticated with LocalCloudManager's certificate and its identity token (`/api/plantdescriptions` → `/plantdescriptionengine/mgmt/plant-descriptions`).

A companion *Service consumers* view lists, per provider and service definition, the consumers authorised by **ConsumerAuthorization** and those orchestrated by **DynamicServiceOrchestration** and **SimpleStoreService Orchestration**, so that engineered connections and access rules can be compared. Alarms are not yet shown in the UI; they are read through *plantDescriptionMonitor*.

Operation in the test beds. Both 5.2.2 test clouds run the PDE with mutual TLS, outsourced authentication, `management.policy=whitelist` (LocalCloudManager only; operators always) and per-call authorisation of *plantDescriptionMonitor*. The 23 October Java migration replaces the earlier SSO rule backend with DSO managed scopes; PDE no longer needs SimpleStore. Apply the DSO schema upgrade before enabling the new runtime. The PDE needs registry management permission and scoped query/replace grants at DSO, plus the registry's direct-access list; framework systems are exempt from `SYSTEM_NOT_IN_DESCRIPTION`. The PDE is operational in both test clouds; in `Arrowhead-test-cloud` its synchronisation failed until the direct-access list was corrected on 2026-10-15 (Section 13).

6.2.2 Choreographer

Purpose. Executes pre-defined workflows (*plans*) on *executor* systems.

Services. *choreographerSession* (`start-session`, `abort-session`), *choreographerExecutor* (`register`, `unregister`, `notify-step-done`), *choreographerManagement* (14 operations on plans, sessions, steps, worklog, executors), *generalManagement*, *sysMonitor*. It consumes the executor-side interface (`service-info`, `start`, `abort`).

Internal design. A plan is a chain of actions; each action is a DAG of steps (cycle-checked and transitively reduced). A step names a 5.2.2 orchestration requirement, quantity and static parameters. A session runs the plan `quantity` times; independent steps run in parallel on a worker pool (`session.worker.threads`, 8), and step completions are serialised per session. For each step an executor is chosen and **DynamicService Orchestration** *serviceOrchestrationPushManagement* is used to `subscribe` (target = executor) and `trigger`; DSO pushes the result to an internal callback that accepts only DSO's certificate and pending correlation ids, with timeout `orchestration.timeout`. Executor results `SUCCESS`, `ERROR` (retry with another executor), `FATAL_ERROR`, `ABORTED`. Sessions active at restart are aborted. Data: ten tables in `ah_choreographer`.

6.2.3 Configuration

Purpose. Central store of one configuration file per system.

Services. *configuration* (`get-configuration`, `get-raw-configuration`), *configurationProxy* (`get-configuration`, `store-configuration`), *configurationManagement* (`list-configurations`, `store-configuration`, `remove-configuration`), *generalManagement*, *sysMonitor*.

Internal design. A system may read only its own configuration (path name equals the authenticated, certificate-bound name). The proxy requires sysop or a ConsumerAuthorization grant. Content is validated (Base64, media type, plain file name, size $\leq$ `configuration.max.size`) and stored as bytes in `configuration_data`.

6.3 Security and Life Cycle

6.3.1 CertificateAuthority

Purpose. Issues and manages the life cycle of local-cloud end-entity X.509 certificates.

Services. *certificateSigning*, *certificateValidation* (certificate and trusted-key status), *certificateAuthorityManagement* (list/revoke certificates, add/list/delete trusted keys), *sysMonitor*; all HTTPS offers use `CERT_AUTH`.

Internal design. Signing validates the PKCS#10 proof of possession, bounds the validity, replaces any requested CA constraint with `BasicConstraints(ca=false)` and signs with the read-only mounted local-cloud CA key, returning an ordered chain. Validation checks issuer signature, validity and local revocation. Role checks on the client certificate CN: signing only for **Onboarding**, **LocalCloudManager**, `Sysop`, **Certificate Authority**; management without **Onboarding**. Its MySQL schema stores serials, subjects, validity, PEM and revocation time, and trusted public keys with validity intervals. There is no insecure mode. The deployed CA publishes a CRL at startup, after revocation and hourly; relying services and brokers enforce it on fresh TLS handshakes. No OCSP service is claimed [14].

6.3.2 Onboarding

Purpose. Admission control of the local cloud: the complete on- and off-boarding procedure of Bicaku et al. [13] on the 5.2.2 Core. A device that holds only a manufacturer-issued bootstrap certificate, and that an operator approved, obtains step by step an onboarding certificate, a device certificate with its registry entry, and for each approved hosted system a system certificate and an Authentication identity. Off-boarding withdraws all of it. It establishes a chain of trust from the hardware device to its systems and their services.

Services.

- *onboarding*: `onboard-with-csr` and `onboard-with-name` (bootstrap certificate → onboarding certificate, profile `on`, at most 24 h, plus registry descriptors), `onboard-device` (onboarding certificate → device certificate, profile `de`, and **ServiceRegistry** device entry), `onboard-system` (device certificate → system certificate, profile `sy`, and identity with a generated password returned once), `offboard-device` (device certificate).
- *onboardingManagement*: `query-enrollments`, `get-enrollment`, `approve-device` (bootstrap SHA-256 fingerprint and approved system names), `offboard-device`, `remove-enrollment`; requires an operator identity token bound to the operator certificate.
- *generalManagement*, *sysMonitor*.

Both application services use `generic_https` with `CERT_AUTH`; the v4 shared-secret enrolment has been removed.

Internal design.

- **Certificate-only enrolment.** TLS client authentication is `need`; the trust store holds the cloud CA and the approved manufacturer CAs. A bootstrap certificate is accepted only if its fingerprint has been approved; its subject may differ from the device name.
- **Credential chain.** Each step accepts only the credential of the previous step, and only the onboarding and device certificate issued last, and only while **CertificateAuthority** `check-certificate` reports them GOOD. Repeating a step issues a new credential and revokes the previous one.
- **No CA key.** Every certificate is signed by the **CertificateAuthority** (*certificateSigning* `sign`); Onboarding stores no private key and no password.
- **Registry and identities.** *serviceRegistryManagement* `device-create` or `device-update` (a device that already hosts systems cannot be removed and re-created), *identityManagement* `identity-mgmt-create`. The systems register themselves, naming their device.
- **Off-boarding** executes `system-remove`, `device-remove`, `identity-mgmt-remove`, `revoke-certificate` and *blacklistManagement* `create` for every system; it continues after a failed action and reports each one.
- **Data.** `device_enrollment` (device name, bootstrap fingerprint, approved systems, state `APPROVED` → `DEVICE_REGISTERED` → `OFFBOARDED`, identifiers and serials of the latest onboarding and device certificates, approver), `system_enrollment` (system, device, certificate, expiry), `logs`.
- **Required rights** (test clouds: `scripts/configure-onboarding-policy.sh`): grants for certificate signing, validation and revocation, identity creation and removal and blacklist entries; **ServiceRegistry** management whitelist and direct access.

The procedure is verified end to end in both test clouds with a demo device (`make smoke-onboarding-procedure`).

6.4 Information and Messaging

6.4.1 EventHandler

Services. *eventPublish* (`publish`), *eventSubscription* (`subscribe`, `unsubscribe`), *eventHandler Management* (`query-subscriptions`, `update-subscription`, `remove-subscription`), *general Management*, *sysMonitor*.

Internal design. Subscriber and publisher identities come only from the authenticated request. On publish, subscriptions of the normalised event type are selected, filtered by validity window (`startsAt`, `expiresAt`) and by metadata set containment, and each match is delivered asynchronously to its HTTP callback with EventHandler credentials, retried up to `event.delivery.attempts` (default 1). A scheduled job deletes expired subscriptions. Event-type authorisation is not enforced: the ConsumerAuthorization `verify` model does not let an intermediary check on behalf of publisher and subscriber.

6.4.2 DataManager

Services. *dataProxy* and *dataHistorian*, each with `list-systems`, `list-services`, `get-data`, `put-data` on `/datamanager/data-proxy/...` and `/datamanager/data-historian/...`, resource path `/systems/<system>/services/<service>/data`; *sysMonitor*.

Internal design. SenML records (RFC 8428 field names kept) are cached in memory by the proxy and persisted by the historian; historian queries accept `from`, `to`, `count`, `signals`, `signalCount`. With `service.authorization.enabled` every call is certificate-bound and checked with *authorization* `verify`. DataManager stores no *sysMonitor* data; that history belongs to **SysMonitor**.

6.4.3 TimeManager

Services. *time* (`get-time`), *generalManagement*, *sysMonitor*.

Internal design. A built-in SNTP client queries `ntp.server.list` in order every `ntp.check.interval` (60 s), accepting only server-mode replies with stratum 1–15 that echo the originate timestamp. Time is *trusted* while $|offset| <$ `time.offset.threshold` (5 s); it is untrusted before the first check and when no server answers. `get-time` returns epoch, time zone, DST, trust flag, offset, source and last check; `echo` is `degraded` while untrusted.

6.5 Monitoring and Analytics

6.5.1 SysMonitor

Purpose. Collects, validates and stores the *sysMonitor* observations of all microsystems and republishes them in normalised form.

Services. *sysMonitorManagement* (status, providers, subscription policy, reconcile; restricted to **LocalCloud Manager**), *sysMonitorHistory* (`list-systems`, `query`, `latest`; each call checked with *authorization* `verify`), *sysMonitorData* (normalised MQTT stream).

Internal design. Components: lifecycle coordinator, discovery and orchestration adapters, identity/authorisation adapter, metadata compiler, AMQP intake, observation pipeline, observation history, outbox and MQTT data publisher, management API, audit/health. A reconciliation loop refreshes the *sysMonitor* providers from **DynamicServiceOrchestration** every 30 s (so only authorised providers are followed) and binds the durable intake queue `SysMonitor.observations` to the routing keys of the providers and observation types selected through *sysMonitorManagement* (selective subscriptions). Each delivery is checked for broker-session identity and provenance, schema-validated, normalised and de-duplicated by observation id, then written in one database transaction to the history and the outbox; the outbox entry is removed after the *sysMonitorData* PUBACK (at-least-once). Poison messages are quarantined. Retention: `SYSMONITOR_RETENTION_DAYS` (30). One active instance is the baseline.

6.5.2 SysMonitorPublisher (deprecated)

A sidecar that registered and published *sysMonitor* (`publisherMode=delegated`) for systems without a native publisher. Deprecated since 2026-10-11, when all test-cloud systems became native publishers; kept only for third-party systems.

6.5.3 AIGym

Services. *aiGym* (`status`, `add-sample`, `predict`, `observations`) on `generic_https`/`CERT_AUTH`, and *sysMonitor* with AI extension observations.

Internal design. A nearest-neighbour model sandbox with ephemeral, thread-safe in-memory model stores. At start-up and in a 30 s reconciliation loop it registers itself and pulls *sysMonitorData* from **DynamicServiceOrchestration**, subscribing to the normalised monitoring stream. Every request needs a CA-issued certificate, an identity token whose name matches the CN, and a ConsumerAuthorization allow decision.

6.6 Inter-Cloud

6.6.1 Gatekeeper

Services. *globalServiceDiscovery* (`query`, `multi-query`), *interCloudNegotiation* (`negotiate`), *gatekeeperManagement* (`clouds`, `put-clouds`, `delete-cloud`, `relays`, `put-relays`), *echo*, *sysMonitor*. The v4.6 paths (`/gatekeeper/init_gsd`, `init_icn`, `init_multi_gsd`) are kept.

Internal design. Java-Spring service with mandatory mTLS, identity-token/CN binding, ConsumerAuthorization policy and persistent cloud and relay records in its MySQL schema. GSD and ICN are executed with explicitly approved neighbour **Gatekeeper**s over the inter-cloud relay. The Company peer profile is extended by explicit organisation bindings for RustDev/Lab (Section 12); foreign identity is not inferred from a matching local system name. Exchange uses the relay (Section 6.6.3). As provider side, the **Gatekeeper** keeps only the providers that its ConsumerAuthorization grants to the remote consumer and cloud; being an intermediary, it uses *authorizationManagement* `check-policies` with the cloud identifier `<Cloud>|<Organisation>` (granted for that operation only), since `verify` answers only the provider or consumer itself. ICN has the **GateTunnel**s of both clouds open a session; the orchestration result points to the consumer-side **GateTunnel**.

6.6.2 GateTunnel

Services. *gateTunnelConnection* (`connect-provider`, `connect-consumer`, `close-sessions`), *gateTunnelKey* (`public-key`), *gateTunnelManagement* (`sessions`, `close-session`), *echo*, *sysMonitor*; v4 aliases under `/gateway`.

Internal design. Successor of the 4.6 Gateway. Implements the session life cycle (`PROVIDER_READY` → active → closed) and persistent allocation of control ports from 8000–8099. Inter-cloud sessions are opened by the local **Gatekeeper** only (`/gatetunnel/intercloud/provider`, `/gatetunnel/intercloud/consumer`). The consumer side is an HTTPS listener on 8200–8299 for the session's consumer only; requests are forwarded as messages over the relay. The provider side accepts them only from the negotiated peer **Gate**

Tunnel, within the provider's base path and negotiated operation scope, and calls the provider with its own certificate and identity token, recording the original consumer in `X-Arrowhead-Remote-Consumer`. No unauthenticated raw TCP bridge is exposed. The deployed federation profile is AH-GT-1: signed ephemeral X25519 exchange and AES-256-GCM-protected request/reply traffic. The relay does not receive plaintext application bodies. Exact Rust path semantics and the qualified interoperability boundary are described in Section 12.

6.6.3 Inter-Cloud Relay

The relay (`Arrowhead-relay`) is an Eclipse Mosquitto 2.0 MQTT broker with mutual TLS, shared by approved local clouds. The original Company-only profile is extended through explicit RustDev/Lab peer bindings; each cloud retains its own application CA. In the Java/Go test beds it joins their cloud networks as `arrowhead-relay` (port 8883) and is published on the host on 8893.

- **Organisation PKI.** An organisation CA (`Company.arrowhead.eu`, profile `or`) issues the relay's server certificate and a *cross-certificate* for each cloud CA (same subject and key), name-constrained to `C=SE,O=Company,OU=<Cloud>`. Each local cloud keeps its own CA; inside a cloud only that CA is trusted, so a certificate of another cloud is refused. Only the relay and the relay clients of **Gatekeeper** and **GateTunnel** trust the organisation CA.
- **Relay identities.** **Gatekeeper** and **GateTunnel** use dedicated relay certificates (client authentication only) with subject `/C=SE/O=Company/OU=<Cloud>/CN=<System>.<Cloud>.Company.arrowhead.eu/dnQualifier=sy`. Because of the name constraint, a cloud CA can vouch only for its own OU: a certificate with another cloud's OU, or an ordinary cloud certificate, fails verification at the relay.
- **Topics and access list.** Messages use `arrowhead/relay/<targetCloud>/<gatekeeper|gatetunnel>/in/<senderCN>/<messageId>`. The broker takes the full verified subject DN as user name; the access list lets each system read only its own cloud's inbox of its own role and write into another cloud's inbox only below its own CN. The receiver takes the sender's identity and cloud from the topic, never from the message body.
- **Operation.** The clouds are started first, then the relay; the organisation PKI is regenerated after any cloud CA change, and each cloud's `make intercloud` registers the neighbour cloud at its **Gatekeeper** and the inter-cloud grants.

7 Services

All application services are version `1.0.0`. Every Java system also produces *generalManagement* (`get-log`, `get-config`); every project-migrated system produces *sysMonitor* (Section 4.5). These are omitted below.

7.1 Produced Services

Microsystem	Service definition	Operations
Blacklist^{U}	*blacklistDiscovery*	check, lookup
	blacklistManagement	query, create, remove
TranslationManager^{U}	*translationBridge*	discovery, negotiation, abort
	translationBridgeManagement	discovery, negotiation, abort, query
	translationReport	report
DeviceQoSEvaluator^{U}	*qualityEvaluation*	filter, sort
	deviceQualityDataManagement	query, reload
PlantDescriptionEngine	*plantDescriptionManagement*	query, get, create, replace, update, remove
	plantDescriptionMonitor	query-plant-descriptions, get-plant-description, query-alarms, get-alarm, acknowledge-alarm
Choreographer	*choreographerSession*	start-session, abort-session
	choreographerExecutor	register, unregister, notify-step-done
	choreographerManagement	query-plans, get-plan, create-plan, remove-plan, check-plan, start-sessions, abort-session, query-sessions, query-session-steps, query-worklog, query-executors, get-executor, create-executor, remove-executor
Configuration	*configuration*	get-configuration, get-raw-configuration
	configurationProxy	get-configuration, store-configuration
	configurationManagement	list-configurations, store-configuration, remove-configuration
CertificateAuthority	*certificateSigning*, *certificate Validation*, *certificateAuthority Management*	sign; certificate/trusted-key check; list/revoke certificates, add/list/delete trusted keys
Onboarding	*onboarding*	onboard-with-name, onboard-with-csr, onboard-device, onboard-system, offboard-device
	onboardingManagement	query-enrollments, get-enrollment, approve-device, offboard-device, remove-enrollment
EventHandler	*eventPublish*	publish
	eventSubscription	subscribe, unsubscribe
	eventHandlerManagement	query-subscriptions, update-subscription, remove-subscription
DataManager	*dataProxy*, *dataHistorian*	list-systems, list-services, get-data, put-data
TimeManager	*time*	get-time
SysMonitor	*sysMonitorHistory*	list-systems, query, latest
	sysMonitorData	normalised observation stream (MQTT)
	sysMonitorManagement	status, providers, subscriptions, reconcile
AIGym	*aiGym*	status, add-sample, predict, observations
Gatekeeper	*globalServiceDiscovery*	query, multi-query
	interCloudNegotiation	negotiate
	gatekeeperManagement	clouds, put-clouds, delete-cloud, relays, put-relays
GateTunnel	*gateTunnelConnection*	connect-provider, connect-consumer, close-sessions
	gateTunnelKey	public-key
	gateTunnelManagement	sessions, close-session

Table 4: Application services produced by the 5.2 support systems.

7.2 Consumed Services

Beyond registration (*systemDiscovery*, *serviceDiscovery*), identity (*identity*) and management authorisation, which every system consumes as described in [1], the support systems consume:

Consumer	Service & operation (provider)	Purpose
TranslationManager	*serviceDiscovery* lookup (SR); *blacklistDiscovery*; *authorization*/*authorizationToken* (CA); *interface BridgeManagement*: check-targets, initialize-bridge, abort-bridge; data model translator factory: check, initialize	Bridge discovery and negotiation
DeviceQoSEvaluator	*serviceRegistryManagement*: system-query (SR); device agent `/device-qos`	Device map, augmented measurements
PlantDescriptionEngine	*serviceRegistryManagement*: system-query, service-query (SR); *serviceOrchestrationPlant Management*: query, replace (DSO); *sysMonitor History* (SysMonitor)	Rule synchronisation, monitoring alarms
Choreographer	*serviceOrchestrationPushManagement*: subscribe, trigger (DSO); *serviceDiscovery* lookup (SR); executor `service-info`, `start`, `abort`	Orchestration for executors, plan check, step execution
Configuration, **Data Manager**, **TimeManager**	*authorization*: verify (ConsumerAuthorization)	Per-call authorisation in the default secure profile
Onboarding	*certificateSigning* sign, *certificateValidation* check-certificate, *certificateAuthorityManagement* revoke-certificate (CertificateAuthority); *identity Management* identity-mgmt-create/-remove (Authentication); *serviceRegistryManagement* device-create/-update/-remove, system-remove and *serviceDiscovery* lookup (SR); *blacklistManagement* create (Blacklist)	On- and off-boarding steps
EventHandler	subscriber callback URIs (HTTP POST)	Event delivery
SysMonitor	*serviceDiscovery* (SR); *serviceOrchestration* pull (DSO); *authorization* verify; every authorised *sys Monitor* (AMQP)	Provider discovery, history access control, collection
AIGym	*serviceOrchestration* pull (DSO); *sysMonitorData* (SysMonitor)	Monitoring data for experiments
Gatekeeper, **GateTunnel**	*authorization* verify (own services); **Gatekeeper**: *authorizationManagement* check-policies, *service Discovery* lookup, neighbour **Gatekeeper**s; **Gate Tunnel**: peer **GateTunnel**, the provider's service; both over the relay (MQTT)	Inter-cloud discovery, negotiation and sessions
DynamicService Orchestration (Core)	*blacklistManagement* query; *translationBridge Management* discovery, negotiation; *quality Evaluation*	Orchestration enrichment (Core GSoSDD, Fig. 5)
All `arrowhead-common` systems	*blacklistDiscovery*: check	Inbound blacklist filter, if enabled

Table 5: Services consumed by the 5.2 support systems.

8 Use Cases (White Box)

8.1 Device Onboarding and Off-boarding

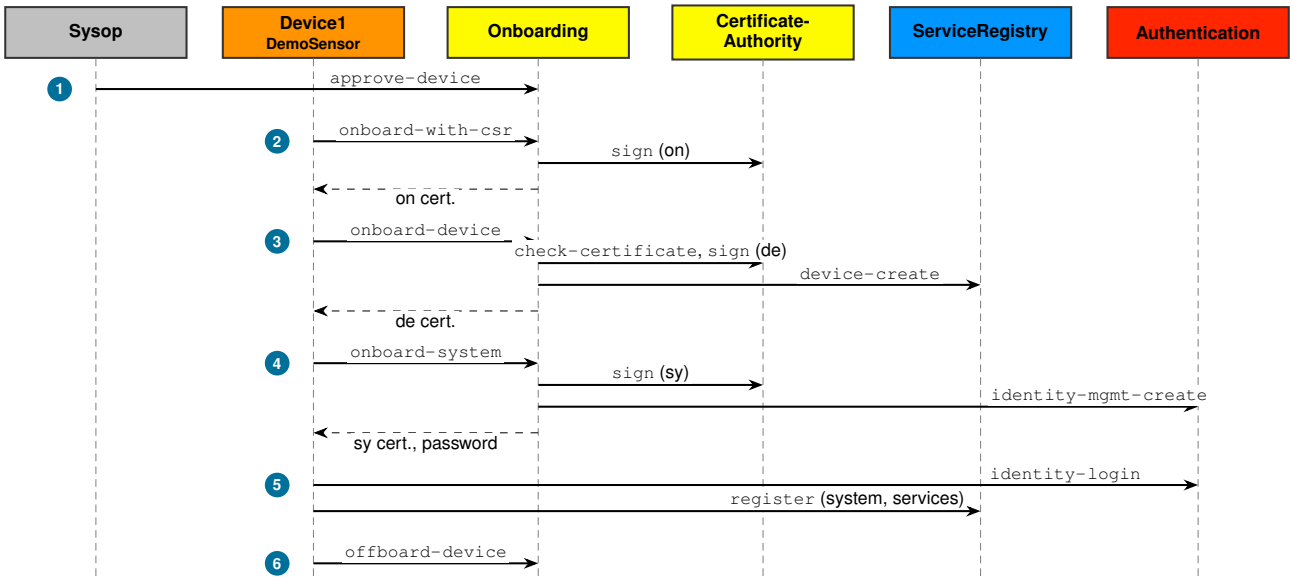


Figure 7: The 5.2.2 onboarding procedure, as exercised by `make smoke-onboarding-procedure`.

① The operator approves `Device1`: the fingerprint of its bootstrap certificate and the system `DemoSensor`. ② The device presents the bootstrap certificate and receives an onboarding certificate (`on`), signed by the **CertificateAuthority**, and the registry descriptors. ③ With it, the device receives its device certificate (`de`) after a status check, and is registered as a device. ④ With the device certificate, it receives a system certificate (`sy`) and an identity for `DemoSensor`. ⑤ `DemoSensor` logs in with its own certificate and registers itself and its services. ⑥ Off-boarding removes the registry entries and identity, revokes the certificates and blacklists the system. This realises the procedure of [13] and of the 5.0 Support GSoSD [3], except that the announcement steps are replaced by pre-distributed addresses.

8.2 Plant-Description-Driven Orchestration

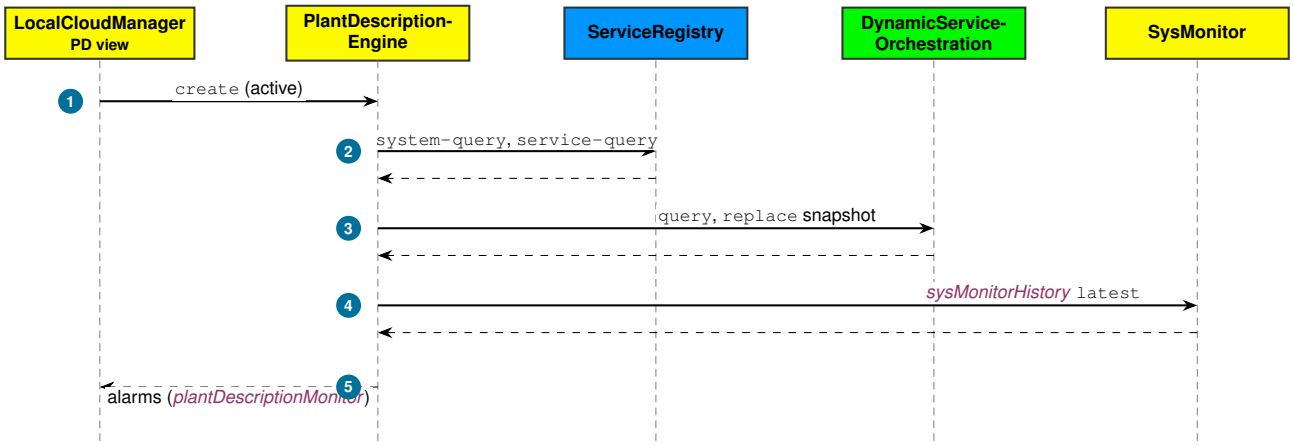


Figure 8: Synchronisation loop of the PlantDescriptionEngine (every 15 s and after each change).

① An engineer creates or edits a plant description in the PD view of the LocalCloudManager console and

activates it; the console stores it with *plantDescriptionManagement*. ② The PDE matches plant systems to registered systems. ③ It reconciles its complete active-plant snapshot; consumers then pull their orchestration from **DynamicServiceOrchestration** (managed plant scopes plus normal security checks). ④ Liveness of plant systems is read from the observation history. ⑤ Deviations become alarms, read through *plantDescription Monitor*. Prerequisites: DSO plant-management extension deployed; PDE in the management whitelists of SR and scoped query/replace rights at DSO and in the SR direct-access list; LocalCloudManager in the PDE's management whitelist; a grant for *sysMonitorHistory*.

8.3 Workflow Execution by the Choreographer

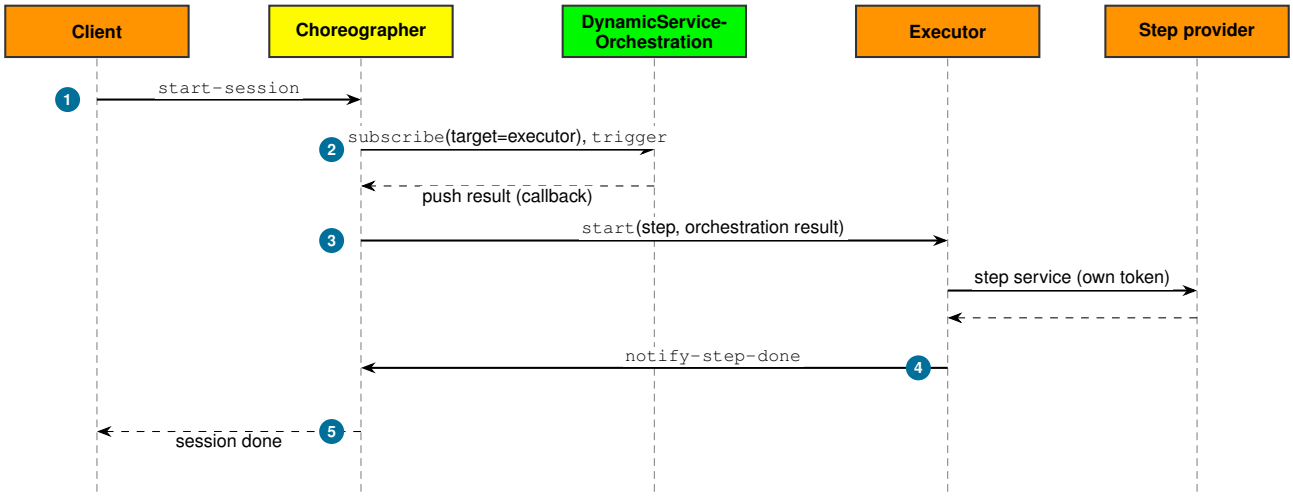


Figure 9: Execution of one plan step.

① A client starts a session by plan name. ② For each ready step the Choreographer chooses an executor and lets DSO orchestrate the step's service requirement *for the executor*, so authorisation and tokens are evaluated for the executor. ③ The executor is started with the result and ④ reports completion (accepted only from that executor). ⑤ After the last action and iteration the session ends.

8.4 Monitoring Data Path

Covered by Figure 2: native publication by every system, orchestrated and authorised subscription by **Sys Monitor**, durable history, and consumption through *sysMonitorHistory* (PDE alarms) and *sysMonitorData* (AIGym, analytics).

8.5 Inter-Cloud Service Consumption

① RandomDataProducer orchestrates *dataHistorian* `put-data` with the flag `ONLY_INTERCLOUD`; **Dynamic ServiceOrchestration** has `enable.intercloud=true`. ② It calls its **Gatekeeper**'s *globalServiceDiscovery*; the request reaches the other cloud's **Gatekeeper** over the relay, which keeps only the providers its Consumer-Authorization grants to that consumer of that cloud. ③ *interCloudNegotiation* checks authorisation again and has both **GateTunnel**s open the session; the orchestration result is the consumer-side listener, with cloud identifier `<ProviderCloud>|Company`. ④ RandomDataProducer calls the listener with its own certificate and token; the request crosses the relay, and the provider **GateTunnel** calls DataManager with its own certificate and token, which therefore needs the `put-data` grant. Verified in both directions between `TestCloud` and `GoCloud`.

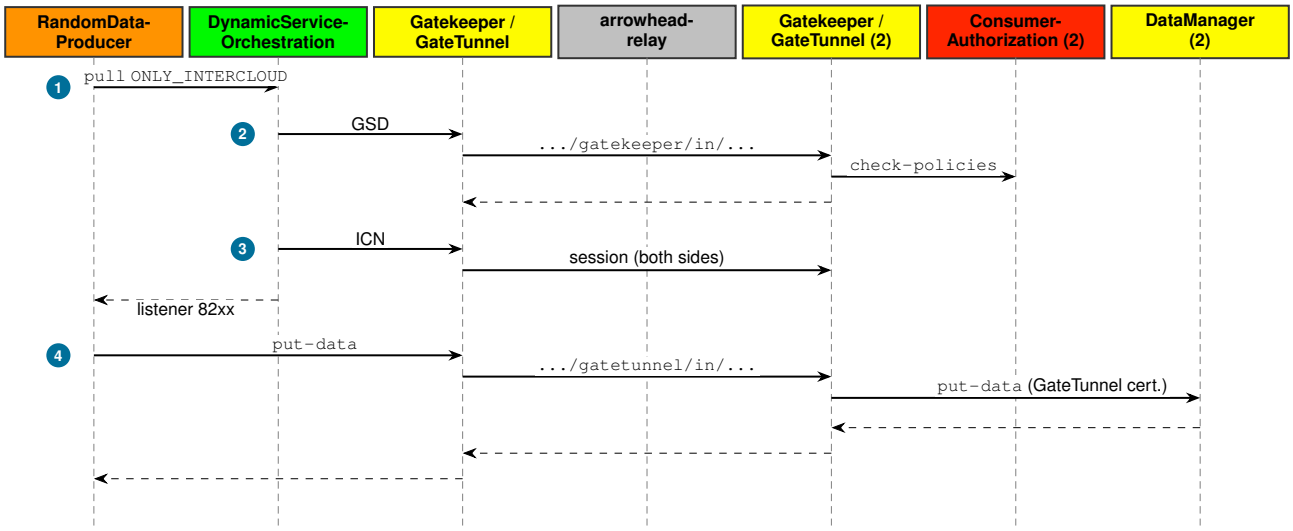


Figure 10: RandomDataProducer of one test cloud storing data at the other cloud's DataManager.

8.6 Plant-Managed Dynamic Orchestration (Local Extension)

The Java implementation dated 23 October 2026 changes the PDE backend from SimpleStore to **Dynamic ServiceOrchestration**. PDE atomically publishes the complete active-plant snapshot using *serviceOrchestration PlantManagement*: `query` is `POST/serviceorchestration/mgmt/plant-rules/query`; `replace` is `PUT/serviceorchestration/mgmt/plant-rules/replace`. This is a local Java extension; it is not part of the unmodified upstream 5.2.2 contract.

A scope is a consumer and service definition with ranked service-instance identifiers. Lower nonnegative priority wins among eligible providers. Empty provider lists retain managed scopes with no result; an empty scope list clears the active plant. DSO first restricts candidates to the engineered set, then retains its normal authorisation, blacklist, lock, interface and QoS checks. With matchmaking, it selects the best surviving plant priority; without matchmaking it returns the survivors in priority order. Consumer preferences can narrow this set, but cannot introduce an unengineered provider. Managed scopes cannot escape through local failure, `ALLOW_INTERCLOUD` or `ONLY_INTERCLOUD`.

The `orchestration_plant_rules` table stores one atomic snapshot in DSO's database, retaining it across restarts. Replacement requires both management-policy permission and an authenticated PDE or Sysop identity; LocalCloudManager may query only. PDE retains its own plant-description and alarm stores. An unresolved producer produces an empty managed scope, rather than deleting the consumer's engineering constraint. No active description clears the snapshot. Query/replacement failures leave PDE degraded rather than claiming that the plant is in force.

Apply `database_patch_pde_dso.sql` to an existing DSO database before starting the upgraded runtime. The Java test-cloud migration target creates the table and grants data access, and configures operation-scoped query/replace permissions. Verify PDE's successful synchronisation before relying on the new constraints. Existing consumer orchestration caches (normally 300 s) can delay visible changes beyond the PDE's 15 s interval. Legacy PDE-owned SSO entries are not automatically deleted; optional cleanup must preserve operator-owned SSO rules. SimpleStore remains an independent optional backend, but PDE no longer requires it. This update records source/configuration changes, not a live deployment or cross-language conformance result.

9 Security

9.1 Default Secure Profile and Development Exceptions

Full authentication and authorisation (AA) and encryption of exchanged information are the default from v5.0 for Support systems as well as Core and application systems. Support services therefore join an already protected

cloud: trusted credentials, Authentication and ConsumerAuthorization readiness, encrypted service interfaces and operation-scoped access rights are prerequisites, not optional additions after registration. Onboarding may provision approved credentials but does not itself permit application operations.

The Java reference profile uses HTTPS with mutual TLS and certificate-bound identity tokens. Native *sysMonitor* uses AMQPS with mutual X.509 authentication (SASL `EXTERNAL`); *sysMonitorData* uses MQTTS. The inter-cloud relay uses MQTTS, and AH-GT-1 additionally encrypts application bodies end to end between GateTunnels. TLS-only broker protection must not be confused with such end-to-end payload protection, or with encryption of persistent application records.

Component switches for plaintext transport, self-declared identities or disabled service authorisation are explicit isolated-development exceptions and do not redefine the architectural default. No support system has an insecure default mode. Missing credentials, trust or a required AA decision must fail closed rather than trigger an unsecured fallback. The Core GSoSDD's bootstrapping section gives the corresponding security-readiness sequence.

9.2 Security Model

- **Authentication**: X.509 certificates of the local cloud (profile `SYSTEM`; `ON_BOARDING` only for **Onboarding** output), combined in the test clouds with `OUTSOURCED` identity tokens whose system name must equal the certificate CN.
- **Authorisation of produced services**: `CERT_AUTH` interface policy plus *authorization* `verify` per call (cached 30 s, fail closed) for **TimeManager**, **DataManager**, **Configuration** (proxy), **PlantDescription Engine** (monitor), **SysMonitor** (history), **AIGym**, **Gatekeeper** and **GateTunnel**; ownership rules (own configuration, own session, own step, own bridge report) where the identity of the caller is decisive.
- **Management**: `/mgmt/` paths protected by the common management filter; the test clouds use `sysop-only`, except where a management client is whitelisted (LocalCloudManager at the **Plant DescriptionEngine**; the PDE and **Onboarding** at the **ServiceRegistry**). **CertificateAuthority** and **SysMonitor** apply CN allowlists.
- **Admission control**: **Onboarding** issues credentials only to operator-approved devices and their approved systems, through a chain in which each step requires the previous credential, and withdraws them by off-boarding. It adds a security mechanism before authentication and authorisation: it decides whether a credential exists at all (GSoSD [2], Section 4.2).
- **Management layer**: the engineering UI acts only through *plantDescriptionManagement* with LocalCloud-Manager's own certificate and token; the console requires operator login, host/origin validation and a constrained Docker proxy (Section 4.7).
- **Key material**: the cloud CA key is used only by the **CertificateAuthority**, mounted read-only; **Onboarding** holds no signing key; server-side `ca=false` enforcement on all issued certificates.
- **Database privileges**: each database-backed system connects with its own MySQL account, restricted to its own schema (Section 2.3). At run time a system only reads and changes rows: JPA uses `ddl-auto=validate`, no system runs schema migrations or DDL, Quartz keeps its jobs in memory, and the Log4j2 JDBC appender only inserts into `logs`. The application account therefore needs only `SELECT`, `INSERT`, `UPDATE` and `DELETE` on the system's own tables. Tables are created and migrated by the initialisation scripts under a separate administrative account, which the application never receives. Since 8 November 2026 every source grant script of the Core and support systems in the primary Java tree grants exactly these four privileges. In the independent Compose deployments the MySQL image first grants the `MYSQL_USER` account `ALL` on the schema; an initialisation step (`least-privilege.sh`) then revokes it and grants the four privileges. A compromised system can therefore read and change its own rows but cannot drop, alter or truncate tables or create objects. Databases initialised before this change keep their old grant until an operator applies the same restriction as root (Section 13).
- **Telemetry**: the AMQP broker's access list limits each publisher to its own routing keys and **SysMonitor** to binding the cloud's *sysMonitor* keys; the MQTT broker's access list limits **SysMonitor** to writing its data namespace; replay resistance through observation ids, sequences and timestamp skew bounds.

- **Inter-cloud trust**: organisation CA with cross-certificates name-constrained to each cloud; dedicated relay identities for **Gatekeeper** and **GateTunnel**; relay access list per verified subject DN; provider-side authorisation with `check-policies` at discovery and negotiation; inside each cloud only its own CA is trusted.

10 Non-Functional Requirements as Realised

Caching. Authorisation decisions are cached for 30 s in the support systems that verify per call; the DataManager proxy caches the latest SenML records; **SysMonitor** provides a local history so consumers need not query providers.

Relative system independence. Each system owns its store; support-system failure does not stop the Core. Liveness is reported, not enforced.

Backwards compatibility. The migrations are deliberately not wire-compatible with 4.6: URL paths are kept where possible, but payloads, identities (names instead of numeric ids), credentials and management APIs follow 5.x. The old deployment must be kept for rollback until clients are migrated.

Naming convention. PascalCase systems, camelCase services, kebab-case operations, snake_case interface templates throughout [8]; SenML field names are kept as the governing external standard.

11 Port Allocation

Port numbers follow the *Port Numbering Specification* of the Core GSoSDD [1], which is normative for both documents. In short: each Arrowhead framework system type has one canonical port, unique across the framework; a local cloud uses the block 8000–8999, divided into functional ranges; local clouds that share hosts or addresses are given a cloud index $k \in \{0, \ldots, 23\}$ and use $canonical + 1000 \cdot k$. Table 6 shows the part of the allocation that concerns the support systems.

Range	Use	Support system ports
8000–8099	**GateTunnel** control port pool	dynamic
8200–8299	**GateTunnel** inter-cloud session listeners	dynamic
8430–8439	Life cycle, engineering	8432 **PlantDescriptionEngine**, 8435 **Onboarding**
8440–8449	Core and trust infrastructure	8448 **CertificateAuthority**, 8449 **Gatekeeper**
8450–8469	Support systems	8453 **GateTunnel**, 8455 **EventHandler**, 8457 **Choreographer**, 8459 **Configuration**, 8461 **DataManager**, 8463 **TimeManager**, 8464 **Blacklist**, 8465 **Translation Manager**
8470–8479	Monitoring, QoS, analytics	8472 **DeviceQoSEvaluator**, 8475 **SysMonitor**, 8478 **AIGym**
8480–8489	Health endpoints	8480 **Gatekeeper**, 8481 **GateTunnel**, 8482 **SysMonitor** (allocated; not used by the Java-Spring reference)
8490–8499	Operator tools	8490/8491 LocalCloudManager console/API, 8492 **LocalCloudConfigurator** (8493 host worker), 8494 **SoSConfigurator** (8495 console)
8800–8899	Messaging infrastructure	8883 MQTT broker (TLS): *sysMonitorData* and other MQTT traffic
(5671)	AMQP broker (TLS), well-known port outside the block	all *sysMonitor* traffic

Table 6: Support-system part of the canonical port allocation ($k = 0$).

The 5.2.2 test beds form a small system of local clouds on one host: `Arrowhead-test-cloud` uses $k = 0$, `Arrowhead-test-cloud-go` $k = 1$ (host ports 9xxx, MQTT 9883, AMQP 6671) and `Arrowhead-test-cloud-rust` $k = 2$ (host ports 10xxx, e.g. LocalCloudManager UI 10490). Systems listen on their canonical ports inside each cloud's container network and only the published host ports carry the offset. The inter-cloud relay belongs to neither cloud: it listens on 8883 inside both cloud networks and is published on the host on 8893. Since v0.3 both test beds conform: the random-data-producer test application

uses 8500 from the application range (it previously shared **TimeManager**'s 8463), **DataManager** runs on 8461 (was 8462), and the LocalCloudManager UI and API use 8490 and 8491 (were 8080 and 8790).

12 Interoperability with the Java Reference Implementation

Interoperability is assessed at an explicit service-contract boundary, independently of implementation language. Three different claims must be distinguished: *core-slot substitution* within one local cloud, *federated service consumption* between independently trusted clouds, and *mixed-language providers* registered in the same local cloud. Passing one boundary does not prove complete replacement of every Java API, persistent state format or operational behaviour.

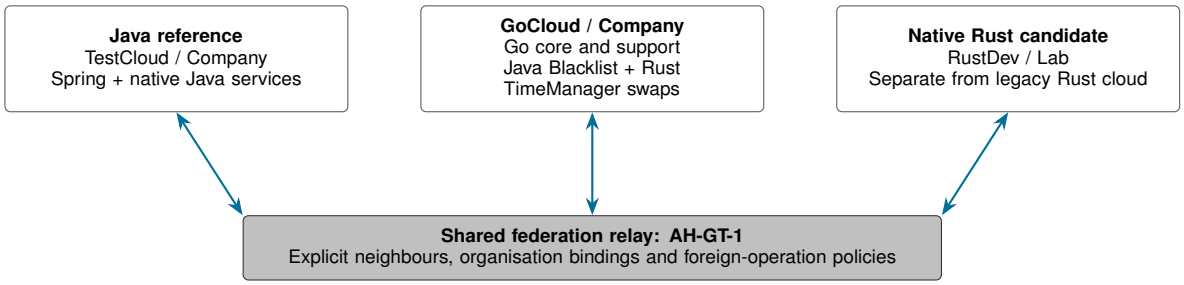


Figure 11: Implementation and trust boundaries. Federated clouds keep their own identities and state. GoCloud additionally demonstrates Java and Rust support providers under its own local-cloud identity and trust configuration.

12.1 Evidence and Current Deployment Scope

Five-core substitution (19–20 October). The Java-cloud interchangeability rerun placed Go and Rust implementations in the same five service slots as Java: ServiceRegistry, Authentication, ConsumerAuthorization, DynamicServiceOrchestration and SimpleStoreServiceOrchestration. The three transitions Go to Rust, Rust to Java and Java to Go passed the recorded core behavioural checks and the same ten Java support-client checks. DNS names, ports, cloud CA and system certificates were retained; each implementation kept separate persistent state. Corrections included the narrowly scoped identity bootstrap lookup, unrestricted bootstrap-discovery metadata, certificate-bound registration/discovery and Java-compatible identity service paths [18]. This dated sequence is evidence of the tested boundary, not an assertion about the currently selected image or database migration.

Native Rust federation and monitoring (19 October). The native `arrowhead-rust-candidate` deployment, identified as `RustDev|Lab`, is distinct from the older `arrowhead-v52-test-cloud-rust` deployment. Its native producer/historian path and AMQP–SysMonitor–MQTT–AIGym pipeline have functional evidence. AH-GT-1 global discovery, negotiation and encrypted HTTP forwarding were exercised with Java and Go peers, including historian read-back, operation/path denial and identity-binding checks. Cross-organisation exchange requires an explicit `RustDev=Lab` peer binding, approved neighbour records and scoped foreign-consumer policies; it does not merge cloud CAs, accounts or business data [16].

Plant-managed orchestration (23 October). GoCloud and the native Rust candidate now implement the Java project's *serviceOrchestrationPlantManagement* extension. Each passed 19 live extension checks, including atomic replacement, invalid-request rejection, persistence, priority/fallback behaviour, unavailable scopes and restriction under both inter-cloud flags. Both final snapshots were empty because no operator plant was active. The legacy Rust cloud has no PDE and was not part of this update. The consoles passed read checks; the report does not establish newly enabled console mutation rights [19].

Mixed-language and MQTT translation (24 October). GoCloud now exposes the Java-compatible MQTT API from all five Go core systems and registers `generic_mqtts`. Reversible swaps run the Java Blacklist with its MQTT API and a native Rust TimeManager with an HTTPS-only API against the Go core. The Go TranslationManager and **TestTranslation** exercise `ALLOW_TRANSLATION` with HTTPS/MQTT interface bridging and SenML-to-`simpleJson` data-model conversion. Recorded paths include MQTT consumption of the Go

DataManager, HTTPS consumption of the Go DSO MQTT API, direct/bridged calls to Java Blacklist, and MQTT consumption of Rust TimeManager through a bridge. Wrong bridge tokens, usage reporting and abort were checked; the reported regression passed both with the swaps active and after restoration [20].

Interpreting the source-contract audit. The 23 October audit compared 284 registered Java operations with native registrations and reported 256 exact Go matches and 229 Rust matches. Those are historical service/operation/method/path counts, not current equivalence scores: the 24 October MQTT/translation changes and newer runtime registrations were not re-audited here. The audit identified path-parameter conventions, renamed management operations, policy/version metadata and LocalCloudManager contract differences; DTO fields and error bodies were not measured. These remain acceptance dimensions rather than claims closed by the passing workload tests [21].

12.2 Contract and Security Requirements

A replacement must preserve the service definition and version, operation names, DTO fields and validation, interface-template properties, policy labels, discovery metadata and externally advertised base paths. HTTPS and MQTT are separate bindings: a compatible HTTPS endpoint alone does not imply MQTT compatibility. MQTT requests retain authentication, blacklist, management and authorisation checks, and responses are confined to the authenticated system's response topic. Identity login/logout/change remain HTTPS-only in the tested Go MQTT profile [20].

The broker certificate is also part of the cross-language contract. GoCloud now issues system-profile subjects for `AmqpBroker.GoCloud.Company.arrowhead.eu` and `MQTTBroker.GoCloud.Company.arrowhead.eu`, while retaining DNS/IP SANs. This satisfies Rust's explicit broker-identity pinning as well as Java/Go/Python hostname verification. Rust TimeManager publishes its native monitoring through AMQP with `messaging.amqpIdentity=AmqpBroker`; it remains an HTTPS-only application provider [20].

12.3 Boundaries and Acceptance of Further Replacements

The evidence establishes the recorded paths, not universal Java equivalence. TestTranslation is a test provider with bounded interface/data-model support; it does not demonstrate arbitrary translators or translation chains. The Rust candidate still lacks complete Java DTO/error/side-effect coverage and complete onboarding provisioning/renewal/off-boarding, advanced choreography, remaining QoS/translation/provider-MQTT integrations and full lifecycle qualification. Its segment-exact tunnel path matching is intentionally stricter than reference descendant matching. The 23 October PDE/DSO update supersedes the older Rust plant-synchronisation limitation, but does not close these other gaps [20, 16, 19].

For each further substitution, record the exact service operations and transports, implementation build, cloud identity, state store and policies. Verify positive consumer/provider calls and denial cases, including token/certificate mismatch, revocation on fresh connections, unauthorised operations and paths, broker topic isolation where applicable, persistence/restart behaviour and rollback to the previous implementation. Stop the replaced service before starting its successor to avoid duplicate identities, DNS aliases and ports. Java Blacklist has its own MySQL store and Rust TimeManager its own state volume; existing Go state is retained, not automatically converted. Passing these checks is functional acceptance within the recorded scope, not production or security certification.

For Go, remaining qualification includes QoS selection, complete push/recovery semantics, golden DTO/error fixtures and state migration. The current support compatibility notes still distinguish SysMonitor's MQTT 3.1.1 output from Java MQTT 5 metadata semantics; the new core MQTT API and translation tests do not by themselves close that telemetry binding gap. Conversely, older catalogue entries claiming no Go MQTT, no Go translation, no native monitoring, no federation or no Go CRL enforcement must not override newer code and dated evidence. The Go security source now defines fail-closed configured CRL checks, and native Rust still documents missing MQTT service APIs and remaining QoS/translation pipelines [22, 20, 16].

13 Implementation Observations and Open Issues

1. **DSO plant-management extension.** PDE now uses the local DSO managed-rule API, removing its SSO requirement. The DSO schema migration and scoped grants must be installed before startup. Go and native Rust gained the matching extension on 23 October; upstream release and wider conformance remain open (Section 12).
2. **Default port 8465 (resolved in v0.2)** was used by **SimpleStoreServiceOrchestration**, **Translation Manager** and **Onboarding**. **Onboarding** now uses 8435 and **SimpleStoreServiceOrchestration** 8442; the remaining port non-conformances of the test beds are listed in Section 11.
3. **sysMonitor depends on an unreleased common-library extension.** The native publisher and the *sys Monitor* contract exist only in the workspace; the upstream v5.2.2 systems still register the older *monitor* service (`echo`). Upstreaming the contract (SD/IDD and publisher) would make the monitoring chain release-ready.
4. **Restricted discovery and management whitelists must be extended manually** for **Choreographer**, **PlantDescriptionEngine** and **Onboarding** (`service.discovery.direct.access`, `management.whitelist`). The lists are easy to break: in `Arrowhead-test-cloud` the direct-access list lost the PDE and the Choreographer when Onboarding was added, and the PDE's synchronisation failed (*service definition not found: serviceRegistryManagement*) until it was corrected on 2026-10-15.
5. **Management roles and established sessions.** The Java console now requires login, host/origin checks and a scoped Docker proxy. Engineering/operator separation remains a design question. CRLs protect fresh TLS handshakes; off-boarding must also end identity sessions and established connections. These deployment controls supersede the earlier anonymous-console and certificate-only revocation findings [14].
6. **Path-exact management authorisation in DSO** makes `unsubscribe/{id}` effectively sysop-only; **Choreographer** therefore lets push subscriptions expire instead of removing them.
7. **Event-type authorisation** cannot be enforced by **EventHandler** with the current *authorization* `verify` model (intermediary problem).
8. **Federation qualification.** AH-GT-1 now encrypts relayed application traffic and binds it to negotiated identities/operations. Interoperability is proven for the recorded Java/Go/Rust paths, not exhaustive DTO/path semantics. Rust applies stricter segment-exact path grants; organisational relay-server CRLs and full lifecycle qualification remain open [16].
9. **Two brokers per local cloud**: *sysMonitor* uses AMQP, *sysMonitorData* and other MQTT traffic use MQTT. Whether to converge these transports remains open. Deployed cloud-issued certificate revocation is checked; this does not establish an organisation-wide relay certificate lifecycle.
10. **Documentation inconsistency**: the CertificateAuthority SysD speaks of five registered service definitions but lists four.
11. **Ephemeral AIGym models** and single-instance designs (**CertificateAuthority**, **SysMonitor**) limit availability; replication needs shared stores.
12. **Database account privileges (resolved in v0.14).** The source scripts and the Compose database images now restrict application accounts to `SELECT`, `INSERT`, `UPDATE` and `DELETE`; this was checked on fresh databases for all fifteen systems. Volumes of already deployed clouds keep `ALL PRIVILEGES` until an operator applies the restriction, and the historical trees used by deployed clouds are not changed (Section 9.2).
13. **Release status**: the 13 project migrations should be reviewed by the Roadmap WG for inclusion in an upstream support-system release.

14 References

[1] J. Delsing, "Generic System of Systems Design Description: Arrowhead Core Systems 5.2.2 – White Box, v1.1," 2026, roadmap/5.2/Arrowhead SoS architecture/WhiteBox_5.2.2.

[2] ——, “Generic System-of-Systems Description (GSoSD): The Arrowhead Core and Support Systems 5.2.2, v2.1,” 2026, roadmap/5.2/Arrowhead SoS architecture/BlackBox_5.2.2/GSoSD-Core-and-Support-Systems-5.2.2.

[3] Eclipse Arrowhead Roadmap WG, “Generic System-of-Systems Description (GSoSD): The Arrowhead Support Systems 5.0 (draft 0.1),” 2023, roadmap/5.2/Arrowhead SoS architecture/OLD/GSoSD Arrowhead Support Systems 5.0.docx.

[4] Eclipse Arrowhead, “ah5-blacklist-java-spring, release v5.2.2,” https://github.com/eclipse-arrowhead/ah5-blacklist-java-spring, 2026.

[5] ——, “ah5-translation-manager-java-spring, release v5.2.2,” https://github.com/eclipse-arrowhead/ah5-translation-manager-java-spring, 2026.

[6] ——, “ah5-device-qos-evaluator-java-spring, release v5.2.2,” https://github.com/eclipse-arrowhead/ah5-device-qos-evaluator-java-spring, 2026.

[7] EISLAB, Luleå University of Technology, “ah5-support-java-spring: Arrowhead 5.2.2 support system migrations from v4.6 (workspace), with per-system SysD, SysDD, SD and IDD,” 2026, not an upstream release.

[8] Eclipse Arrowhead Roadmap WG, “Naming convention for local clouds, microsystems and microservices (Naming_convention_2),” https://github.com/eclipse-arrowhead/roadmap, 2025.

[9] ——, “Generic System-of-Systems Description (GSoSD): The Arrowhead Core Systems 5.0, v1.2,” 2025, roadmap/5.2/Arrowhead SoS architecture/BlackBox_5.2.2/GSoSD Arrowhead Core Systems 5.0 v1.2.docx.

[10] C. Jennings, Z. Shelby, J. Arkko, A. Keranen, and C. Bormann, “RFC 8428: Sensor measurement lists (senml),” IETF, 2018.

[11] D. Mills, “RFC 4330: Simple network time protocol (sntp) version 4,” IETF, 2006.

[12] M. Nystrom and B. Kaliski, “RFC 2986: Pkcs #10: Certification request syntax specification,” IETF, 2000.

[13] A. Bicaku, S. Maksuti, C. Hegedűs, M. Tauber, J. Delsing, and J. Eliasson, “Interacting with the Arrowhead Local Cloud: On-boarding Procedure,” in *Proc. 2018 IEEE Industrial Cyber-Physical Systems (ICPS)*, 2018.

[14] EISLAB Arrowhead workspace, “Java TestCloud deployment and orchestration snapshot,” 2026, 24 October. Arrowhead-test-cloud-java: compose.yaml, README.md and service-policy scripts; live Docker project arrowhead-v52-test-cloud, 22 microsystems and 41 containers.

[15] EISLAB, Luleå University of Technology, “SysMonitor-AMQP-IDD-5.2.2: sysMonitor observation contract over AMQP 0-9-1,” 2026, ah5-support-java-spring/SysMonitor/Document/LaTeX.

[16] EISLAB Arrowhead workspace, “Native Rust federation and functional acceptance,” 2026, 19 October. Arrowhead-test-cloud-rust/candidate-native/FUNCTIONAL-FEDERATION.md and REPLACEMENT-STATUS.md; later PDE results supersede the earlier plant-synchronisation limitation.

[17] Eclipse Arrowhead model library, “Arrowhead-5.2 Papyrus model: Network, Device and generic local-cloud templates,” Local Papyrus project, EclipseArrowhead-5.0-JD, 2026, arrowhead-5.2.uml, Network model and applied ArrowheadDSL stereotypes; Arrowhead-5.2.notation, Network diagram. Inspected 10 November 2026. Network classifiers and protocol choices are library definitions, not a deployed address or firewall inventory.

[18] EISLAB Arrowhead workspace, “Interchangeability rerun, 19–20 October 2026,” 2026, arrowhead-test-cloud-java/interchangeability/RESULTS-2026-10-19.md. Five core slots, three interleaved implementation transitions and unchanged Java support clients.

[19] ——, “Go and native Rust PDE/DSO deployment,” 2026, 23 October. security-evaluation/pde-dso-go-rust-2026-10-23/README.md. Nineteen live extension checks passed in each cloud; console verification was read-only.

[20] ——, “GoCloud: MQTT API, translation bridges, Java and Rust support systems,” 2026, 24 October. Arrowhead-test-cloud-go/MQTT-2026-10-24.md; companion TRANSLATION-2026-10-24.md. Dated engineering test evidence.

[21] ——, “Arrowhead 5.2.2 Java, Go and Rust interchangeability,” 2026, 23 October. Local source-contract comparison, retained in the Interchangability analysis directory. Historical operation counts predate the 24 October MQTT/translation updates; DTO and error-body parity were not measured.

[22] ——, “Native implementation compatibility and security boundaries,” 2026, sources inspected 24 October: ah5-core-go/docs/COMPATIBILITY.md and SECURITY.md; ah5-support-go/COMPATIBILITY.md; ah5-core-rust/COMPATIBILITY.md; ah5-support-rust/COMPATIBILITY.md. Later dated deployment evidence takes precedence over superseded gap lists.

15 Revision History

15.1 Amendments

No.	Date	Rev.	Subject of Amendments	Author
1	2026-10-13	0.1	Initial white-box GSoSDD of the 5.2 support systems (upstream v5.2.2 and EISLAB project migrations).	Jerker Delsing
2	2026-10-13	0.2	Added Section 11 (port allocation per the Core GSoSDD port numbering specification); Onboarding moved from 8465 to 8435.	Jerker Delsing
3	2026-10-13	0.3	Test beds made conformant with the port specification (DataManager 8461, TimeManager 8463, random-data-producer 8500, LocalCloudManager 8490/8491).	Jerker Delsing
4	2026-10-15	0.4	PlantDescriptionEngine operational with its first engineering UI (PD view of LocalCloudManager) as management layer; Onboarding rewritten for the complete on- and off-boarding procedure as admission control; services, use cases, security model and open issues updated.	Jerker Delsing
5	2026-10-15	0.5	sysMonitor over AMQP (broker, routing keys, access list, publishers; *sysMonitorData* on MQTT); inter-cloud relay (organisation PKI, relay identities, topics, access list), Gatekeeper and GateTunnel internals, inter-cloud use case, ports (GoCloud AMQP 6671, GateTunnel pool 8000–8099, Rust cloud $k = 2$), security and open issues.	Jerker Delsing
6	2026-10-23	0.6	PDE backend migrated from SSO to DSO managed snapshots; updated dependencies, use cases, security constraints and migration requirements.	Codex (draft)
7	2026-10-24	0.7	Added Java deployment and service-path figures; scoped Java/Go/Rust interoperability evidence through 24 October, MQTT translation, native PDE/DSO and current deployment/security corrections.	Codex (draft)
8	2026-10-24	0.8	Applied the SysML v1.6 Papyrus library colour convention to all system diagrams, sequence participants and deployment graphics; added a shared palette and legend.	Codex (draft)
9	2026-10-24	0.9	Rewrote bootstrapping and corresponding security descriptions: full AA and encrypted information exchange are the default from v5.0; unsecured test modes are explicit exceptions.	Codex (draft)
10	2026-10-25	0.10	Separated four native Java support projects into independent build and deployment units, with private helpers, configuration, credentials and state; added deployment boundary figure.	Codex (draft)
11	2026-10-25	0.11	Extended independent packaging to all fifteen Java support systems, private common/DTO source, own DB/configuration and isolated deployment checks; documented the non-Java console exception.	Codex (draft)
12	2026-10-30	0.12	Updated primary Java/Go/Rust packaging and evidence limits, four-core configuration default, graphical engineering tools and integrated private database ownership.	Codex (draft)

13	2026-11-08	0.13	Database privileges: least-privilege rule for application database accounts in the security model and data-ownership section; current grants and Compose behaviour recorded as an open issue.	Claude (draft)
14	2026-11-08	0.14	Database privileges narrowed in the Java source scripts and Compose database images; security model and open issue updated.	Claude (draft)
15	2026-11-08	0.15	Native Java systems paragraph made a subsubsection of the implementation patterns; duplicated packaging statements replaced by a reference to the independent deployment section.	Claude (draft)
16	2026-11-09	0.16	All support systems Java-Spring: CertificateAuthority, Gatekeeper, GateTunnel, SysMonitor, LocalCloudManager, LocalCloudConfigurator and SoSConfigurator converted (MySQL storage, unchanged contracts); inventory, implementation patterns, independent deployments, consoles and port allocation (8492–8495) updated.	Claude (draft)
17	2026-11-10	0.17	Network deployment from Arrowhead-5.2 Papyrus: segment/endpoint mapping, restricted flows and commissioning evidence; model definitions distinguished from deployment decisions.	Codex (draft)

15.2 Quality Assurance

No.	Date	Version	Approved by
1		0.5	